PROGRAMMING LANGUAGES

TypeScript

From tsconfig and strict mode to conditional types, Zod-validated boundaries, React and Node typing, incremental migration and build performance at scale.

typescript · tsconfig · strict-mode · generics · conditional-types · discriminated-unions · narrowing · utility-types · zod · runtime-validation · declaration-files · react-typescript · js-to-ts-migration · project-references · fde

Study Pack · Comprehensive reference & interview prep · ~107 min read · 23,546 words

CONTENTS

1. Why TypeScript, and When Plain JS Is Fine
2. Setup: tsc, tsconfig and Bundlers
3. Primitives, Literals, Arrays, Tuples and the Three Special Types
4. Interfaces vs Type Aliases
5. Unions, Intersections and Discriminated Unions
6. Narrowing and Control-Flow Analysis
7. Functions: Overloads, Generics and satisfies
8. Generics in Depth
9. Utility Types
10. Modules, Declaration Files and Typing Third-Party JS
11. Async Code: Promises, fetch and Error Typing
12. Runtime Validation with Zod
13. Typing React Properly
14. Typing a Node Backend
15. Errors You Will Hit and How to Read Them
16. Migrating a JavaScript Codebase
17. Performance and Scale
18. Interview Questions
19. Checklist and Practice Plan

Why TypeScript, and When Plain JS Is Fine

TypeScript is JavaScript with a static type system bolted on at build time and erased at run time. That single sentence explains both its enormous value and every frustrating thing about it. As a Forward Deployed Engineer you will meet TypeScript in three places — the customer's frontend, their Node services, and the glue code you write to integrate with their APIs — and in all three, the types are the fastest way to understand a codebase you have never seen.

0
TYPES SURVIVE TO RUNTIME

~15%
OF BUGS CAUGHT BY TYPES ALONE (ROUGH, DEBATED)

#1
REASON TEAMS ADOPT IT: REFACTORING CONFIDENCE

strict
THE ONE SETTING THAT DECIDES EVERYTHING

What you actually get

REFACTORING WITHOUT FEAR

Rename a field on a widely used object and the compiler lists every place that must change. In a 200-file JavaScript codebase that same rename is a grep and a prayer. This is the benefit teams cite most, and it is the one that compounds.

DOCUMENTATION THAT CANNOT ROT

A function signature tells you what it needs and returns, and the compiler enforces that the signature is true. Comments lie after six months; types cannot, because the build fails.

EDITOR INTELLIGENCE

Autocomplete that knows your domain, inline errors as you type, go-to-definition that works across packages. On an unfamiliar codebase this is how you orient in hours instead of days.

What you do not get

✕ **Types do not exist at runtime**

TypeScript compiles to JavaScript by deleting the types. A value that arrives from `fetch`, `JSON.parse`, `process.env`, a form, or a database driver is typed by your *assertion*, not by any check. If the API returns `{"id": "42"}` and you declared `id: number`, TypeScript is perfectly happy and your arithmetic silently produces `"421"`. Every boundary where data enters your program needs runtime validation — usually Zod. This is the single most important thing to understand about TypeScript and the most common cause of production bugs in "fully typed" codebases.

When plain JavaScript is the right call

Situation	Verdict
A 60-line build script or one-off migration	Plain JS. The setup costs more than the bugs it prevents.
A throwaway prototype you will delete in a week	Plain JS, or TS with `strict: false` if the team already has a template.
A team with no TS experience and a two-week deadline	Plain JS. Learning curve plus deadline is how you ship nothing.
Anything multi-person and longer than a month	TypeScript.
A library other people will consume	TypeScript, absolutely — your types are your API documentation.
A codebase you will hand over and never see again	TypeScript. The types are the handover document.

✦ **The middle path most people forget**

You can get a large fraction of the benefit without renaming a single file: add `// @ts-check` at the top of a `.js` file, or set `checkJs: true` in `tsconfig.json`, and write types in JSDoc comments. The editor and `tsc` will check them. This is an excellent option when the customer's team is nervous about a migration, because it is reversible and requires no build change at all.

JAVASCRIPT

```javascript
// @ts-check
/**
 * @param {string} customerId
 * @param {{ from: Date, to: Date, limit?: number }} options
 * @returns {Promise<Array<{ id: string, amount: number }>>}
 */
async function fetchOrders(customerId, options) {
  const limit = options.limit ?? 50;
  // The editor now autocompletes options.from and errors on options.form
  return api.get(`/customers/${customerId}/orders`, { ...options, limit });
}
```

Setup: tsc, tsconfig and Bundlers

Most people's first painful day with TypeScript is a configuration problem, not a type problem. The compiler has roughly a hundred flags, about ten of which matter, and getting module resolution wrong produces errors that look like type errors but are not. Learn the ten flags and you will never fight the toolchain again.

Getting started

BASH

```bash
# A Node project
npm init -y
npm i -D typescript @types/node tsx
npx tsc --init                      # generates a heavily commented tsconfig.json

# Compile, type-check, and run
npx tsc                             # emit JS per tsconfig
npx tsc --noEmit                    # type-check only - what you run in CI
npx tsc --watch                     # incremental, on save
npx tsx src/index.ts                # run TS directly, no build step
npx tsx watch src/index.ts          # dev server style reload

# Node 22+ can strip types natively for simple cases
node --experimental-strip-types src/index.ts
```

The tsconfig options that actually matter

JSON

```jsonc
{
  "compilerOptions": {
    /* Language and environment */
    "target": "ES2022",
    "lib": ["ES2022"],
    "module": "NodeNext",
    "moduleResolution": "NodeNext",

    /* Strictness - turn it ALL on */
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "exactOptionalPropertyTypes": true,
    "noImplicitOverride": true,
    "noFallthroughCasesInSwitch": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,

    /* Interop */
    "esModuleInterop": true,
    "allowSyntheticDefaultImports": true,
    "forceConsistentCasingInFileNames": true,
    "resolveJsonModule": true,
    "isolatedModules": true,
    "verbatimModuleSyntax": true,

    /* Emit */
    "outDir": "dist",
    "rootDir": "src",
    "sourceMap": true,
    "declaration": true,
    "declarationMap": true,
    "incremental": true,
    "skipLibCheck": true
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules", "dist"]
}
```

Option	What it does	Why it matters
`strict`	Enables eight other flags including `strictNullChecks` and `noImplicitAny`	Without it TypeScript is a linter with a nice editor. With it, it is a type system. Non-negotiable on new code.
`strictNullChecks`	`null` and `undefined` are not assignable to other types	The entire value proposition. Eliminates the `cannot read property of undefined` class of bug.
`noUncheckedIndexedAccess`	`arr[0]` is `T | undefined`, not `T`	Honest about the fact that indexing can miss. Noisy at first; catches real bugs.
`exactOptionalPropertyTypes`	`{ a?: string }` does not accept `{ a: undefined }`	Distinguishes "absent" from "present but undefined" — matters for JSON and spread merges.
`isolatedModules`	Each file must be transpilable independently	Required by esbuild/SWC/Babel. Forces `export type` for type-only re-exports.
`verbatimModuleSyntax`	Import/export statements are emitted exactly as written	Removes a whole class of "the import disappeared and broke a side effect" confusion.
`skipLibCheck`	Do not type-check `.d.ts` files in `node_modules`	Large build-time win; avoids errors inside dependencies you cannot fix.
`moduleResolution`	How import specifiers map to files	The single most common source of baffling setup errors. See below.

Module resolution, explained once

> ⚠ **The error that wastes an afternoon**
>
> `Cannot find module './utils' or its corresponding type declarations.` The file is right there. The cause is almost always a mismatch between `module` / `moduleResolution` and how your runtime or bundler resolves. Learn the four combinations and the error becomes trivial.

`module` / `moduleResolution`	Use for	Import must look like
`NodeNext` / `NodeNext`	Node, respecting `"type"` in package.json	`./utils.js` — yes, `.js`, even though the file is `.ts`
`ESNext` / `Bundler`	Vite, esbuild, webpack, Next.js	`./utils` — extensionless is fine; the bundler resolves
`CommonJS` / `Node10`	Legacy Node projects	`./utils`
`Preserve` / `Bundler`	Modern "TS checks, bundler builds" setups	`./utils`

TYPESCRIPT

```typescript
// With module: NodeNext and "type": "module" in package.json,
// relative imports need the OUTPUT extension, not the source extension.
import { formatMoney } from "./utils.js";      // correct, even though utils.ts
import type { Order } from "./types.js";

// With moduleResolution: Bundler (Vite, Next.js), this is correct instead:
import { formatMoney } from "./utils";
import type { Order } from "./types";
```

Path aliases

JSON

```
{
  "compilerOptions": {
    "baseUrl": ".",
    "paths": {
      "@/*": ["src/*"],
      "@domain/*": ["src/domain/*"],
      "@test-utils": ["test/utils/index.ts"]
    }
  }
}
```

tsconfig paths do not rewrite your output

`tsc` type-checks `@/domain/order` correctly and then emits `require("@/domain/order")`, which Node cannot resolve at runtime. The compiler does not rewrite the specifier. You need a bundler that understands the alias (Vite, webpack, tsup, esbuild with a plugin), or `tsc-alias` as a post-build step, or Node's own `imports` field with `#`-prefixed subpath imports. Teams lose hours to this because the type-check passes and only the runtime fails.

JSON

```
{
  "name": "my-app",
  "type": "module",
  "imports": {
    "#domain/*": "./dist/domain/*.js",
    "#lib/*": "./dist/lib/*.js"
  }
}
```

Bundlers and runners

Tool	Type-checks?	Use for
tsc	Yes	The source of truth for type checking. Run `tsc --noEmit` in CI regardless of what builds your code.
esbuild	No — strips types	Extremely fast builds. Pair with a separate `tsc --noEmit`.
SWC	No	Same role as esbuild; used by Next.js and Jest transforms.
Vite	No (esbuild/Rollup under the hood)	Frontend dev server and build. Add `vite-plugin-checker` if you want in-editor type errors in the overlay.
tsup	Optional (`--dts`)	Building a library: ESM + CJS + `.d.ts` in one command.
tsx	No	Running a `.ts` file directly in development. Replaces ts-node for most uses.

✦ **The modern split that confuses newcomers**
Transpiling and type-checking are now separate jobs. Your bundler strips types in milliseconds and never checks them; `tsc --noEmit` checks them and emits nothing. That means **your dev server will happily run code with type errors**. This is a feature — fast feedback — but it means CI must run `tsc --noEmit` as a separate step or type errors will reach main.

JSON

```
{
  "scripts": {
    "dev": "vite",
    "build": "tsc --noEmit && vite build",
    "typecheck": "tsc --noEmit",
    "typecheck:watch": "tsc --noEmit --watch",
    "lint": "eslint . --max-warnings 0",
    "test": "vitest run"
  }
}
```

Primitives, Literals, Arrays, Tuples and the Three Special Types

The base of the type system is small, and three of its members — `any`, `unknown` and `never` — carry almost all the subtlety. Getting those three right separates people who write TypeScript from people who write JavaScript with annotations.

Primitives and inference

TYPESCRIPT

```
// Let inference do the work. Explicit annotations here add noise, not safety.
const name = "Pranit";          // type: "Pranit"  (literal, because const)
let city = "Pune";              // type: string    (widened, because let)
const count = 42;               // type: 42
const active = true;            // type: true
const nothing = null;           // type: null
const missing = undefined;      // type: undefined
const big = 90071992547409 93n; // type: bigint
const sym = Symbol("key");      // type: symbol

// Annotate where inference cannot help: function parameters and empty containers
function greet(who: string): string {
  return `Hello, ${who}`;
}
const ids: string[] = [];       // without this it is never[], and push fails
```

Literal types and widening

TYPESCRIPT

```ts
type Status = "draft" | "published" | "archived";

let s1 = "draft";               // string — widened
const s2 = "draft";             // "draft" — not widened

function publish(status: Status) { /* ... */ }
publish(s2);                    // OK
publish(s1);                    // ERROR: string is not assignable to Status

// Three ways to stop widening
const config1 = { status: "draft" as const };        // status: "draft"
const config2 = { status: "draft" } as const;        // whole object deeply readonly
const config3: { status: Status } = { status: "draft" };

// as const is the idiomatic way to build a lookup table AND its union type
const ROLES = ["admin", "editor", "viewer"] as const;
type Role = typeof ROLES[number];                    // "admin" | "editor" | "viewer"

const HTTP_STATUS = {
  OK: 200,
  NOT_FOUND: 404,
  SERVER_ERROR: 500,
} as const;
type StatusCode = typeof HTTP_STATUS[keyof typeof HTTP_STATUS];   // 200 | 404 | 500
```

Arrays, readonly arrays and tuples

TYPESCRIPT

```ts
const names: string[] = ["a", "b"];
const nums: Array<number> = [1, 2];                // identical; style preference

// readonly prevents mutation through this reference
const frozen: readonly string[] = ["a", "b"];
// frozen.push("c");                               // ERROR
const frozen2: ReadonlyArray<string> = ["a"];      // same thing

// Tuples: fixed length, position-specific types
type Point = [x: number, y: number];               // named elements are docs only
type Entry = [key: string, value: unknown];
type Range = [min: number, max: number, inclusive?: boolean];

const p: Point = [10, 20];
const [x, y] = p;                                  // x: number, y: number

// Variadic tuples
type Head<T extends readonly unknown[]> = T extends readonly [infer H, ...unknown[]] ?
type Tail<T extends readonly unknown[]> = T extends readonly [unknown, ...infer R] ? R

type A = Head<[1, 2, 3]>;                          // 1
type B = Tail<[1, 2, 3]>;                          // [2, 3]

// This is how useState gets its type
function useLocal<T>(initial: T): [T, (next: T) => void] {
  let value = initial;
  return [value, (next) => { value = next; }];
}
```

⚠ **`noUncheckedIndexedAccess` changes array access**

With the flag on, `names[0]` has type `string | undefined`, because TypeScript cannot know the array is non-empty. This is *correct* and initially annoying. Handle it with a guard, `at()` plus a check, or destructuring with a default. The flag catches a real bug class — iterating to `length` and off-by-one indexing — and is worth the noise on new code. Enabling it on a large existing codebase will produce hundreds of errors, so do that incrementally.

Enums vs union literals

TYPESCRIPT

```ts
// Numeric enum — avoid. Values are numbers, reverse-mapped, and it emits runtime code
enum Direction { Up, Down, Left, Right }       // Up === 0

// String enum — acceptable, still emits an object
enum Status { Draft = "draft", Published = "published" }

// const enum — inlined, no runtime object, but breaks with isolatedModules
const enum Level { Low = 1, High = 2 }

// Union of literals — the idiomatic modern choice
type Status2 = "draft" | "published" | "archived";

// With a const object when you need the values at runtime
const Status3 = {
  Draft: "draft",
  Published: "published",
  Archived: "archived",
} as const;
type Status3 = (typeof Status3)[keyof typeof Status3];
```

	String enum	Union of literals
Runtime footprint	Emits a JS object	Zero
Works with `isolatedModules`	Yes (not `const enum`)	Yes
Accepts a raw string	No — must use `Status.Draft`	Yes — `"draft"` works directly
Serialises naturally to JSON	Yes for string enums	Yes
Iterate all values	`Object.values(Status)`	Needs an `as const` array
Nominal-ish (not interchangeable with string)	Yes	No

Practical guidance

Default to union literals plus an `as const` object when you need the values at runtime. Use a string enum when you genuinely want the nominal behaviour — stopping someone passing an arbitrary string — and the team already uses them. Never use numeric enums in new code: `Direction.Up === 0` means `if (dir)` is false for `Up`, which is a real bug that has shipped many times.

`any`, `unknown` and `never`

	`any`	`unknown`	`never`
Meaning	"Stop checking this"	"Could be anything; prove it first"	"This cannot happen"
Assign anything *to* it	Yes	Yes	No (nothing is assignable to never)
Assign it *to* anything	Yes	No — must narrow first	Yes (never is assignable to everything)
Access properties	Yes, unchecked	No, until narrowed	N/A
Use for	Escape hatch during migration, marked with a TODO	Anything from `JSON.parse`, `catch`, a network boundary	Exhaustiveness checks, functions that never return

TYPESCRIPT

```ts
// any is contagious — it disables checking everywhere it flows
const data: any = JSON.parse(raw);
data.user.profile.name.toUpperCase();     // compiles. Crashes at runtime.
const n: number = data.anything;          // compiles. Probably wrong.

// unknown forces you to prove the shape
const safe: unknown = JSON.parse(raw);
// safe.user;                             // ERROR — good
if (typeof safe === "object" && safe !== null && "user" in safe) {
  // narrowed enough to access .user, though you still need validation for depth
}

// never: the exhaustiveness check that makes unions safe to extend
type Shape =
  | { kind: "circle"; radius: number }
  | { kind: "square"; side: number }
  | { kind: "rect"; w: number; h: number };

function area(s: Shape): number {
  switch (s.kind) {
    case "circle": return Math.PI * s.radius ** 2;
    case "square": return s.side ** 2;
    case "rect":   return s.w * s.h;
    default: {
      // If someone adds a new member to Shape, s is no longer never
      // and THIS LINE fails to compile. That is the whole point.
      const exhaustive: never = s;
      throw new Error(`Unhandled shape: ${JSON.stringify(exhaustive)}`);
    }
  }
}

// never as a return type: this function does not return normally
function fail(message: string): never {
  throw new Error(message);
}

// never in a union disappears
type T = string | never;                  // string
```

✓ **The exhaustiveness pattern is the highest-value idiom in TypeScript**

Add a case to a union — a new payment method, a new event type, a new error code — and every `switch` that handles it exhaustively fails to compile, pointing you at exactly the code that needs updating. This is the refactoring superpower that justifies the whole language. Write the `const exhaustive: never` default in every switch over a union.

Interfaces vs Type Aliases

This is the most-asked beginner question and the answer is less dramatic than the debate suggests: they overlap heavily, each can do a couple of things the other cannot, and consistency within a codebase matters more than the choice. Knowing the two genuine differences is enough.

The overlap

TYPESCRIPT

```typescript
interface User {
  id: string;
  name: string;
  email?: string;              // optional
  readonly createdAt: Date;    // cannot be reassigned after construction
}

type UserT = {
  id: string;
  name: string;
  email?: string;
  readonly createdAt: Date;
};

// Both work for objects, both support generics, both support extension.
interface Admin extends User { permissions: string[]; }
type AdminT = UserT & { permissions: string[] };
```

The two real differences

Capability	`interface`	`type`
Describe an object or class shape	Yes	Yes
Union types	**No**	Yes
Primitives, tuples, mapped, conditional, template literal types	**No**	Yes
Declaration merging (reopen and add members)	Yes	**No**
`implements` on a class	Yes	Yes (if it is an object type)
Error messages	Usually show the interface name	Sometimes expand to the full structure
Performance on very large shapes	Marginally better (cached)	Marginally worse (recomputed intersections)

TYPESCRIPT

```typescript
// Only `type` can do these
type Id = string | number;
type Coords = [number, number];
type Keys<T> = keyof T;
type Getter<T> = { [K in keyof T as `get${Capitalize<string & K>}`]: () => T[K] };

// Only `interface` can do this - declaration merging
interface Window { myAppVersion: string; }
interface Window { myAppFlags: Record<string, boolean>; }
// Window now has both. This is how you extend global/third-party types.
```

✦ **A rule you can defend in review**

Use `interface` for object shapes that are part of a public API or that someone might need to augment (React props, config objects, library surfaces). Use `type` for everything else — unions, function types, mapped and conditional types, and anything derived. If the team already has a convention, follow it; this is not worth a debate.

Index signatures and Record

TYPESCRIPT

```typescript
// Index signature: arbitrary string keys
interface Headers {
  [header: string]: string;
}

// Record is usually clearer and composes better
type Headers2 = Record<string, string>;
type RoleFlags = Record<"admin" | "editor" | "viewer", boolean>;
// { admin: boolean; editor: boolean; viewer: boolean } - exhaustive, checked

// Mixing an index signature with known keys requires compatibility
interface Config {
  name: string;
  version: number;
  [key: string]: string | number;      // must include the types of known keys
}

// Partial Record for lookups that may miss
type Cache = Partial<Record<string, User>>;
const cache: Cache = {};
const u = cache["abc"];                 // User | undefined - correct
```

⚠ **Index signatures lie by default**

`Record<string, User>` says every string key yields a `User`, which is false — most keys yield `undefined`. Either enable `noUncheckedIndexedAccess`, which makes the lookup `User | undefined` globally, or use `Partial<Record<string, User>>` explicitly. Code that indexes a map and immediately dereferences the result is one of the most common runtime crashes in "type-safe" codebases.

Structural typing

TypeScript compares types by **shape**, not by name. Two types with identical members are interchangeable even if they are unrelated and declared in different files. This is usually convenient and occasionally dangerous.

TYPESCRIPT

```ts
interface Dog { name: string; legs: number }
interface Table { name: string; legs: number }

function walk(d: Dog) { /* ... */ }
const t: Table = { name: "kitchen", legs: 4 };
walk(t);                                  // compiles. Structurally identical.

// Branded types give you nominal behaviour when it matters
declare const brand: unique symbol;
type Brand<T, B extends string> = T & { readonly [brand]: B };

type UserId  = Brand<string, "UserId">;
type OrderId = Brand<string, "OrderId">;

function asUserId(s: string): UserId { return s as UserId; }
function getUser(id: UserId) { /* ... */ }

const oid = "ord_123" as OrderId;
// getUser(oid);                          // ERROR - exactly what you want
getUser(asUserId("usr_123"));             // OK
```

✓ **Where branding earns its keep**

Any codebase with several kinds of string identifier — `userId`, `orderId`, `tenantId`, `sessionToken` — will eventually pass the wrong one, and the bug is invisible because they are all strings. Branding costs five lines and turns that into a compile error. Also excellent for units (`Minutes` vs `Seconds`), currency amounts (`Cents` vs `Rupees`), and validated-vs-raw strings (`SafeHtml` vs `string`).

Unions, Intersections and Discriminated Unions

Unions are how TypeScript models "one of these", and the discriminated union is the single most useful modelling pattern in the language. Most "TypeScript is awkward here" complaints come from trying to model with optional properties what should be a discriminated union.

Unions and intersections

TYPESCRIPT

```ts
type Id = string | number;                       // either
type Timestamped = { createdAt: Date };
type Authored = { authorId: string };
type Post = Timestamped & Authored & { title: string };   // all of them

// On a union you can only access members present on EVERY member
type A = { kind: "a"; value: string; extra: boolean };
type B = { kind: "b"; value: number };
declare const ab: A | B;
ab.kind;                                         // OK — on both
ab.value;                                        // OK — string | number
// ab.extra;                                     // ERROR — only on A

// Intersections of primitives collapse to never
type Impossible = string & number;               // never
```

The optional-property anti-pattern

TYPESCRIPT

```ts
// BAD: everything optional, no invariants expressed, every access needs a check
interface RequestState {
  loading?: boolean;
  data?: User[];
  error?: Error;
}
// Nothing stops { loading: true, data: [...], error: new Error() } — a nonsense state
// Every consumer writes defensive checks and they all disagree.

// GOOD: a discriminated union. Illegal states are unrepresentable.
type RequestState2 =
  | { status: "idle" }
  | { status: "loading" }
  | { status: "success"; data: User[]; fetchedAt: Date }
  | { status: "error"; error: Error; retryable: boolean };

function render(state: RequestState2) {
  switch (state.status) {
    case "idle":    return "Click to load";
    case "loading": return "Loading…";
    case "success": return `${state.data.length} users`;   // data is guaranteed here
    case "error":   return state.retryable ? "Retry?" : state.error.message;
    default: {
      const _never: never = state;
      return _never;
    }
  }
}
```

✓ **Make illegal states unrepresentable**

This is the sentence to remember. If your type permits a combination that cannot occur in reality, you have pushed work onto every consumer and invited bugs. A discriminated union with a literal `kind` / `status` / `type` field is almost always the fix, and TypeScript's narrowing then gives you perfectly typed access inside each branch with no casts.

Discriminated unions in anger

TYPESCRIPT

```typescript
// Domain events — the shape most backend TS codebases converge on
type DomainEvent =
  | { type: "order.created";   orderId: string; customerId: string; total: number }
  | { type: "order.paid";      orderId: string; paymentId: string; paidAt: Date }
  | { type: "order.shipped";   orderId: string; carrier: string; tracking: string }
  | { type: "order.cancelled"; orderId: string; reason: string; refunded: boolean };

type EventType = DomainEvent["type"];              // the union of all type strings

// Extract one member by its discriminant — extremely useful
type EventOf<T extends EventType> = Extract<DomainEvent, { type: T }>;
type PaidEvent = EventOf<"order.paid">;            // { type: "order.paid"; orderId:

// A fully typed handler map: every event must be handled, with the right payload
type Handlers = {
  [K in EventType]: (event: EventOf<K>) => Promise<void>;
};

const handlers: Handlers = {
  "order.created":   async (e) => { await notify(e.customerId, e.total); },
  "order.paid":      async (e) => { await receipt(e.paymentId, e.paidAt); },
  "order.shipped":   async (e) => { await track(e.carrier, e.tracking); },
  "order.cancelled": async (e) => { if (e.refunded) await refund(e.orderId); },
};

async function dispatch(event: DomainEvent): Promise<void> {
  // The cast is needed because TS cannot prove the correlation across the index,
  // but the DECLARATION of `handlers` above is fully checked, which is what matters.
  const handler = handlers[event.type] as (e: DomainEvent) => Promise<void>;
  await handler(event);
}
```

Result types instead of thrown exceptions

TYPESCRIPT

```ts
type Ok<T>  = { ok: true;  value: T };
type Err<E> = { ok: false; error: E };
type Result<T, E = Error> = Ok<T> | Err<E>;

const ok  = <T,>(value: T): Ok<T>  => ({ ok: true,  value });
const err = <E,>(error: E): Err<E> => ({ ok: false, error });

type ParseError =
  | { kind: "empty" }
  | { kind: "not_json"; raw: string }
  | { kind: "wrong_shape"; issues: string[] };

function parseConfig(raw: string): Result<Config, ParseError> {
  if (!raw.trim()) return err({ kind: "empty" });
  let parsed: unknown;
  try {
    parsed = JSON.parse(raw);
  } catch {
    return err({ kind: "not_json", raw: raw.slice(0, 80) });
  }
  const check = ConfigSchema.safeParse(parsed);
  if (!check.success) {
    return err({ kind: "wrong_shape", issues: check.error.issues.map((i) => i.message)
  }
  return ok(check.data);
}

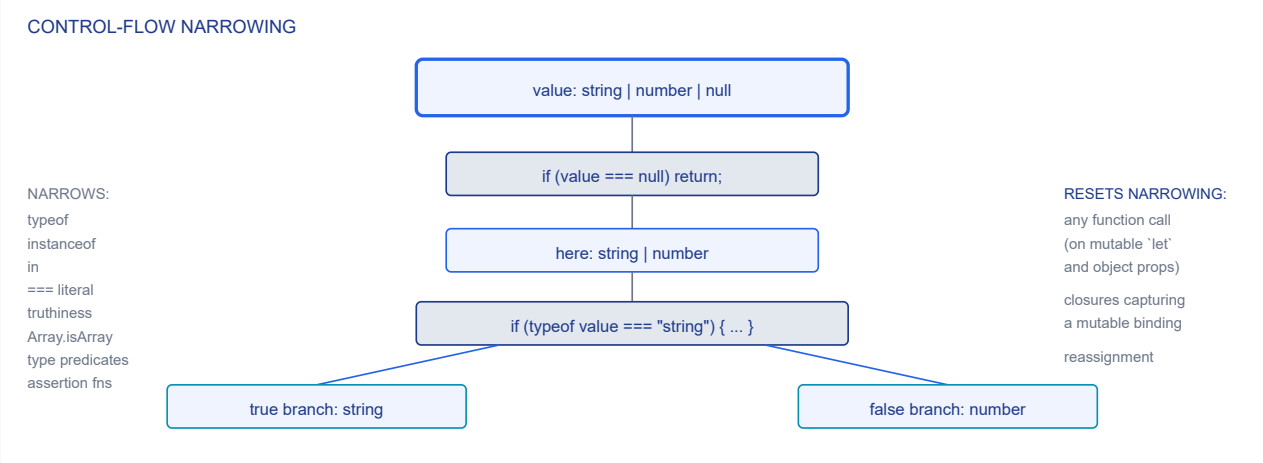
// The caller CANNOT forget to handle the failure — the compiler sees to it
const result = parseConfig(fileContents);
if (!result.ok) {
  switch (result.error.kind) {
    case "empty":       return useDefaults();
    case "not_json":    return fatal(`Config is not JSON: ${result.error.raw}`);
    case "wrong_shape": return fatal(result.error.issues.join("; "));
  }
}
startServer(result.value);              // narrowed to Config here
```

Result types versus exceptions

TypeScript cannot type what a function throws — there is no `throws` clause, and `catch (e)` gives you `unknown`. So exceptions are invisible to the type system, which means the compiler cannot help you handle them. A `Result` type makes failure part of the signature and therefore impossible to ignore. The cost is verbosity and interop friction with libraries that throw. The pragmatic split most teams land on: `Result` for *expected* failures (validation, not-found, business rule violations) and exceptions for *unexpected* ones (programmer errors, infrastructure failure), with one top-level handler.

Narrowing and Control-Flow Analysis

Narrowing is TypeScript watching your code and tracking what a value could be at each point. It is the feature that makes unions pleasant to use rather than a constant source of casts. Knowing exactly which constructs narrow — and the two or three that silently stop narrowing — removes most of the friction people feel with the language.



The compiler tracks a narrowed type per code path. Mutation and function calls can invalidate it.

The narrowing constructs

TYPESCRIPT

```ts
function process(value: string | number | string[] | null | undefined) {
  // 1. typeof — for primitives
  if (typeof value === "string") {
    value.toUpperCase();                // string
  }

  // 2. Array.isArray — special-cased by the compiler
  if (Array.isArray(value)) {
    value.map((s) => s.trim());         // string[]
  }

  // 3. Truthiness — but beware: "" and 0 are falsy
  if (value) {
    // string | number | string[]  — note that "" and 0 are EXCLUDED
  }

  // 4. Explicit null check — usually what you actually meant
  if (value != null) {                  // != null catches both null and undefined
    // string | number | string[]  — "" and 0 are INCLUDED
  }
}

// 5. instanceof — for classes
class HttpError extends Error { constructor(public status: number) { super(); } }
class TimeoutError extends Error {}

function handle(e: unknown) {
  if (e instanceof HttpError)    return `HTTP ${e.status}`;
  if (e instanceof TimeoutError) return "timed out";
  if (e instanceof Error)        return e.message;
  return String(e);
}

// 6. The `in` operator — for objects without a discriminant
type Cat = { meow(): void };
type Dog = { bark(): void };
function speak(pet: Cat | Dog) {
  if ("meow" in pet) pet.meow();
  else pet.bark();
}

// 7. Discriminant equality — the best one
type Shape = { kind: "circle"; r: number } | { kind: "square"; s: number };
function area(sh: Shape) {
  if (sh.kind === "circle") return Math.PI * sh.r ** 2;
  return sh.s ** 2;
}
```

Type predicates (user-defined type guards)

TYPESCRIPT

```typescript
// The return type `x is T` tells the compiler what a true result proves.
function isString(x: unknown): x is string {
  return typeof x === "string";
}

function isNonEmpty<T>(arr: T[]): arr is [T, ...T[]] {
  return arr.length > 0;
}

// Filtering out null — the classic case the compiler cannot infer alone
function isDefined<T>(x: T | null | undefined): x is T {
  return x !== null;
}

const maybe: (User | null)[] = [user1, null, user2];
const users1 = maybe.filter((u) => u !== null);     // (User | null)[] in older TS
const users2 = maybe.filter(isDefined);             // User[] — correct

// TypeScript 5.5+ infers predicates for simple arrow functions:
const users3 = maybe.filter((u) => u !== null);     // User[] in TS 5.5+

// A domain-level guard over unknown data
interface ApiError { code: string; message: string; details?: unknown }

function isApiError(x: unknown): x is ApiError {
  return (
    typeof x === "object" && x !== null &&
    "code" in x && typeof (x as Record<string, unknown>).code === "string" &&
    "message" in x && typeof (x as Record<string, unknown>).message === "string"
  );
}
```

A type predicate is a promise the compiler cannot verify

Nothing stops you writing `function isUser(x: unknown): x is User { return true; }`. The compiler trusts you completely. A wrong predicate is as dangerous as an `as` cast and more insidious because it looks like a check. For data crossing a trust boundary, derive the guard from a schema (Zod's `safeParse`) rather than hand-writing it, so the check and the type cannot drift apart.

Assertion functions

TYPESCRIPT

```typescript
// `asserts x is T` narrows for the REST of the enclosing scope, not just a block.
function assertIsDefined<T>(x: T, name = "value"): asserts x is NonNullable<T> {
  if (x == null) throw new Error(`${name} is required but was ${x}`);
}

function assertNever(x: never, context = "value"): never {
  throw new Error(`Unexpected ${context}: ${JSON.stringify(x)}`);
}

function handler(req: Request) {
  const userId = req.headers.get("x-user-id");     // string | null
  assertIsDefined(userId, "x-user-id header");
  userId.toUpperCase();                            // string, for the rest of the funct
}
```

⚠ **Assertion functions need an explicit type annotation**

You cannot write `const assertIsDefined = (x) => { ... }` and have the assertion work — TypeScript requires the assertion signature to be explicitly declared on an identifier whose type is known at the call site. Use a `function` declaration, or annotate the const explicitly. The error message when you get this wrong ("*Assertions require every name in the call target to be declared with an explicit type annotation*") is one of the less obvious ones.

Where narrowing silently stops

TYPESCRIPT

```ts
class Container {
  value: string | null = null;

  bad() {
    if (this.value !== null) {
      doSomething();                // this call could reassign this.value ...
      // this.value.length;         // ERROR in some configurations - narrowing lost
    }
  }

  good() {
    const v = this.value;           // capture into a const FIRST
    if (v !== null) {
      doSomething();
      v.length;                     // still string - a const cannot change
    }
  }
}

// Closures over a mutable binding also lose narrowing
function example(x: string | undefined) {
  if (x === undefined) return;
  // x is string here
  const later = () => x.toUpperCase();     // OK - x is a parameter, never reassigned
}

function example2(x: string | undefined) {
  if (x === undefined) x = "default";      // reassignment inside the function
  const later = () => x.toUpperCase();     // OK in modern TS, but fragile
}

// Destructuring a discriminated union BREAKS the correlation before TS 4.6
type Action = { type: "add"; n: number } | { type: "reset" };
function reduce(action: Action) {
  const { type, ...rest } = action;
  // `type` and `rest` are now independent; narrowing on `type` does not narrow `rest`
  // Narrow on the object FIRST, then destructure inside the branch.
  if (action.type === "add") {
    const { n } = action;                  // n: number - correct
  }
}
```

✦ **The habit that avoids most narrowing pain**

Copy the value into a `const` at the top of the function, narrow the `const`, and work with that. Mutable bindings and object properties are the only things that lose narrowing, and a `const` local is neither. This single habit eliminates the large majority of "but I already checked that!" moments.

Functions: Overloads, Generics and `satisfies`

Function typing is where TypeScript's inference does the most work for you, and where a few specific tools — overloads, generic parameters, and the `satisfies` operator — let you express APIs that would otherwise require casts at every call site.

Basics and inference

TYPESCRIPT

```typescript
// Annotate parameters, let the return type infer unless you want it enforced
function add(a: number, b: number) {
  return a + b;                                    // inferred: number
}

// Optional and default parameters
function greet(name: string, greeting = "Hello", punctuation?: string): string {
  return `${greeting}, ${name}${punctuation ?? "!"}`;
}

// Rest parameters
function sum(...nums: number[]): number {
  return nums.reduce((a, b) => a + b, 0);
}

// Function types
type Comparator<T> = (a: T, b: T) => number;
type AsyncHandler = (req: Request) => Promise<Response>;
type Predicate<T> = (value: T, index: number) => boolean;

// Contextual typing: the parameter types flow IN from the expected type
const byAge: Comparator<User> = (a, b) => a.age - b.age;    // a and b are User, inferr

// void return means "I ignore whatever you return" - this is why forEach accepts anyt
const log: (msg: string) => void = (msg) => msg.length;    // allowed
```

Overloads

TYPESCRIPT

```ts
// Overload signatures: what callers see. Implementation signature: not callable.
function parse(input: string): object;
function parse(input: string, reviver: (k: string, v: unknown) => unknown): object;
function parse(input: Buffer, encoding?: BufferEncoding): object;
function parse(input: string | Buffer, second?: unknown): object {
  const text = typeof input === "string" ? input : input.toString(second as BufferEnco
  return JSON.parse(text, typeof second === "function" ? (second as never) : undefined
}

// The classic DOM example — return type depends on the literal argument
function createElement(tag: "a"): HTMLAnchorElement;
function createElement(tag: "canvas"): HTMLCanvasElement;
function createElement(tag: "input"): HTMLInputElement;
function createElement(tag: string): HTMLElement;
function createElement(tag: string): HTMLElement {
  return document.createElement(tag);
}

const anchor = createElement("a");       // HTMLAnchorElement — .href is available
```

⚠ **Prefer a union or a generic to an overload**

Overloads do not compose well: you cannot spread arguments into them easily, the implementation signature is unchecked against the overloads in some cases, and error messages degrade to "No overload matches this call", which tells the caller nothing about *why*. Reach for a union parameter type or a generic first. Overloads earn their place mainly when the return type genuinely depends on a literal argument, as above.

Generic functions

TYPESCRIPT

```ts
// The type parameter links input to output
function identity<T>(x: T): T { return x; }

function first<T>(arr: readonly T[]): T | undefined { return arr[0]; }

// Constraints: T must have the shape you depend on
function longest<T extends { length: number }>(a: T, b: T): T {
  return a.length ≥ b.length ? a : b;
}
longest("hello", "hi");            // string
longest([1, 2], [1, 2, 3]);        // number[]
// longest(10, 20);                // ERROR — number has no length

// keyof constraint: the classic typed property getter
function pluck<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}
const user = { id: "1", name: "Asha", age: 31 };
const n = pluck(user, "name");     // string
// pluck(user, "nope");            // ERROR — "nope" is not keyof typeof user

// Multiple keys
function pick<T, K extends keyof T>(obj: T, keys: readonly K[]): Pick<T, K> {
  const out = {} as Pick<T, K>;
  for (const k of keys) out[k] = obj[k];
  return out;
}
const slim = pick(user, ["id", "name"]);     // { id: string; name: string }

// Default type parameters
interface Paginated<T, M = { total: number; page: number }> {
  items: T[];
  meta: M;
}
```

✦ **The generic smell test**

A type parameter is only useful if it appears **at least twice** — typically once in the parameters and once in the return type, or twice in the parameters to force them to match. If `T` appears exactly once, you almost certainly wanted a constraint or a union instead. `function log<T>(x: T): void` is just `function log(x: unknown): void` with extra ceremony.

`satisfies`

TYPESCRIPT

```typescript
type Config = Record<string, string | number | boolean>;

// Annotation: checked, but the specific types are LOST
const a: Config = { host: "localhost", port: 5432, tls: false };
// a.port is string | number | boolean — you cannot do a.port + 1

// `as const`: specific types kept, but NOT checked against Config
const b = { host: "localhost", port: 5432, tls: false } as const;
// no error if you typo a key or use a disallowed value type

// `satisfies`: checked AND specific. This is what you wanted all along.
const c = {
  host: "localhost",
  port: 5432,
  tls: false,
} satisfies Config;
c.port.toFixed(0);                // port is number
// c.bad = Symbol();              // would have errored at declaration

// The most common real use: a route or colour map
const routes = {
  home:    "/",
  user:    "/users/:id",
  orders:  "/users/:id/orders",
} satisfies Record<string, `/${string}`>;

type RouteName = keyof typeof routes;        // "home" | "user" | "orders" — preserved
const path = routes.user;                    // "/users/:id" — the literal, not string

// Combine with as const for deep readonly plus checking
const theme = {
  colors: { primary: "#2563eb", danger: "#dc2626" },
  spacing: [0, 4, 8, 16, 32],
} as const satisfies { colors: Record<string, string>; spacing: readonly number[] };
```

`this` typing

TYPESCRIPT

```ts
// Declare what `this` must be — the compiler checks the call site
interface Button { label: string; onClick(this: Button, e: MouseEvent): void }

// Arrow functions capture `this` lexically and cannot have a this parameter
class Counter {
  count = 0;
  // Class property arrow function: `this` is bound, safe to pass as a callback
  increment = () => { this.count++; };
  // Method: `this` depends on the call site, breaks when detached
  decrement() { this.count--; }
}

const c2 = new Counter();
const inc = c2.increment;  inc();            // works
const dec = c2.decrement;  // dec();         // runtime TypeError: this is undefined
// Enable "strictBindCallApply" and TS catches many of these at compile time.
```

Generics in Depth

Everything beyond basic generics — conditional types, `infer`, mapped types, template literal types — forms a small functional programming language that runs at compile time. You do not need to write it often, but you absolutely need to read it, because every library you depend on is full of it, and because the error messages are unintelligible otherwise.

Conditional types

TYPESCRIPT

```typescript
// T extends U ? X : Y — a ternary over types
type IsString<T> = T extends string ? true : false;
type A = IsString<"hello">;        // true
type B = IsString<42>;             // false

type NonNull<T> = T extends null | undefined ? never : T;
type C = NonNull<string | null>;   // string

// DISTRIBUTIVE: when T is a naked type parameter and the input is a union,
// the conditional distributes over each member and the results are unioned.
type ToArray<T> = T extends unknown ? T[] : never;
type D = ToArray<string | number>;       // string[] | number[]   (distributed)

// Wrap in a tuple to DISABLE distribution
type ToArrayNoDist<T> = [T] extends [unknown] ? T[] : never;
type E = ToArrayNoDist<string | number>; // (string | number)[]

// This is exactly how Exclude and Extract work
type MyExclude<T, U> = T extends U ? never : T;
type MyExtract<T, U> = T extends U ? T : never;
type F = MyExclude<"a" | "b" | "c", "a">;        // "b" | "c"
```

⚠ **Distribution surprises everyone once**

`ToArray<string | number>` giving `string[] | number[]` rather than `(string | number)[]` is the most common conditional-type confusion. The rule: distribution happens only when the checked type is a *bare* type parameter. Wrapping both sides in a one-element tuple (`[T] extends [U]`) turns it off, which is what you want for "is this whole union assignable to that".

`infer`

TYPESCRIPT

```typescript
// infer introduces a type variable inside a conditional's extends clause
type ElementOf<T> = T extends readonly (infer E)[] ? E : never;
type A = ElementOf<string[]>;                    // string
type B = ElementOf<readonly [1, "x", true]>;     // 1 | "x" | true

type ReturnOf<T> = T extends (...args: never[]) => infer R ? R : never;
type C = ReturnOf<() => Promise<User>>;          // Promise<User>

type ParamsOf<T> = T extends (...args: infer P) => unknown ? P : never;
type D = ParamsOf<(a: string, b: number) => void>;   // [a: string, b: number]

// Recursive unwrapping - this is essentially the built-in Awaited
type Unwrap<T> = T extends Promise<infer U> ? Unwrap<U> : T;
type E = Unwrap<Promise<Promise<string>>>;       // string

// Constrained infer
type FirstString<T> = T extends [infer H extends string, ...unknown[]] ? H : never;
type F = FirstString<["a", 1, true]>;           // "a"

// Practical: extract the payload type from a Zod-style schema
type InferSchema<S> = S extends { _output: infer O } ? O : never;
```

Mapped types

TYPESCRIPT

```typescript
// Iterate the keys of T and transform each
type Mutable<T> = { -readonly [K in keyof T]: T[K] };      // remove readonly
type Required2<T> = { [K in keyof T]-?: T[K] };            // remove optional
type Nullable<T> = { [K in keyof T]: T[K] | null };
type Stringify<T> = { [K in keyof T]: string };

// Key remapping with `as` (TS 4.1+) - this is where it gets powerful
type Getters<T> = {
  [K in keyof T as `get${Capitalize<string & K>}`]: () => T[K]
};
interface Person { name: string; age: number }
type PersonGetters = Getters<Person>;
// { getName: () => string; getAge: () => number }

// Filter keys by mapping unwanted ones to never
type OnlyFunctions<T> = {
  [K in keyof T as T[K] extends (...a: never[]) => unknown ? K : never]: T[K]
};
type NonFunctionKeys<T> = {
  [K in keyof T]: T[K] extends (...a: never[]) => unknown ? never : K
}[keyof T];

// Deep variants - note the recursion and the array handling
type DeepPartial<T> = T extends (infer U)[]
  ? DeepPartial<U>[]
  : T extends object
    ? { [K in keyof T]?: DeepPartial<T[K]> }
    : T;

type DeepReadonly<T> = T extends (infer U)[]
  ? readonly DeepReadonly<U>[]
  : T extends object
    ? { readonly [K in keyof T]: DeepReadonly<T[K]> }
    : T;
```

Template literal types

TYPESCRIPT

```ts
type Protocol = "http" | "https";
type Url = `${Protocol}://${string}`;
const good: Url = "https://example.com";          // OK
// const bad: Url = "ftp://example.com";          // ERROR

// Combinatorial expansion
type Size = "sm" | "md" | "lg";
type Variant = "primary" | "secondary";
type ButtonClass = `btn-${Variant}-${Size}`;      // 6 literal members

// Built-in string manipulation types
type U = Uppercase<"hello">;                      // "HELLO"
type L = Lowercase<"HELLO">;                      // "hello"
type C = Capitalize<"hello">;                     // "Hello"
type Un = Uncapitalize<"Hello">;                  // "hello"

// Parse a route and extract its parameters - the pattern behind typed routers
type PathParams<P extends string> =
  P extends `${string}:${infer Param}/${infer Rest}`
    ? Param | PathParams<`/${Rest}`>
    : P extends `${string}:${infer Param}`
      ? Param
      : never;

type Params = PathParams<"/users/:userId/orders/:orderId">;   // "userId" | "orderId"

function buildPath<P extends string>(
  pattern: P,
  params: Record<PathParams<P>, string>
): string {
  return pattern.replace(/:(\w+)/g, (_, k: string) =>
    (params as Record<string, string>)[k]);
}

buildPath("/users/:userId/orders/:orderId", { userId: "1", orderId: "9" });
// buildPath("/users/:userId", { wrong: "1" });   // ERROR - caught at compile time

// Typed event names
type EventName = `on${Capitalize<"click" | "focus" | "blur">}`;  // "onClick" | ...
type Handlers = { [E in EventName]?: (e: Event) => void };
```

Template literal types explode combinatorially

`` `${A}-${B}-${C}` `` where each union has 50 members produces 125,000 types. The compiler has a hard limit (around 100,000 union members) and will error with "*Expression produces a union type that is too complex to represent*". Worse, before hitting the limit it will make your editor sluggish for everyone on the team. Keep template literal unions to small, bounded sets.

Recursive types

TYPESCRIPT

```typescript
// JSON - the canonical recursive type
type Json =
  | string | number | boolean | null
  | Json[]
  | { [key: string]: Json };

// A tree
type TreeNode<T> = { value: T; children: TreeNode<T>[] };

// Dot-notation paths into a nested object - useful for typed config/i18n lookups
type Paths<T, Depth extends readonly unknown[] = []> =
  Depth["length"] extends 5                    // depth guard: stop at 5 levels
    ? never
    : T extends object
      ? { [K in keyof T & string]:
            T[K] extends object ? K | `${K}.${Paths<T[K], [...Depth, 1]>}` : K
        }[keyof T & string]
      : never;

interface Settings {
  app: { name: string; theme: { mode: "light" | "dark"; accent: string } };
  limits: { maxUsers: number };
}
type SettingPath = Paths<Settings>;
// "app" | "app.name" | "app.theme" | "app.theme.mode" | "app.theme.accent"
// | "limits" | "limits.maxUsers"
```

Always put a depth guard on a recursive type

Without the `Depth` tuple accumulator above, a cyclic type (a parent that references a child that references the parent) sends the compiler into infinite recursion and you get "*Type instantiation is excessively deep and possibly infinite*". The tuple-length trick is the standard counter in type-level code: push a dummy element each recursion and check `["length"]`.

Utility Types

TypeScript ships about twenty built-in type helpers that compose into most of what you need. Knowing them cold means you stop re-declaring variants of the same interface, and it makes you much faster at reading library types.

The ones you will use weekly

TYPESCRIPT

```typescript
interface User {
  id: string;
  name: string;
  email: string;
  age?: number;
  readonly createdAt: Date;
}

type A  = Partial<User>;                  // every property optional
type B  = Required<User>;                 // every property required
type C  = Readonly<User>;                 // every property readonly
type D  = Pick<User, "id" | "name">;      // { id: string; name: string }
type E  = Omit<User, "createdAt">;        // everything except createdAt
type F  = Record<"a" | "b", number>;      // { a: number; b: number }
type G  = Exclude<"a" | "b" | "c", "a">;  // "b" | "c"
type H  = Extract<"a" | "b", "a" | "z">;  // "a"
type I  = NonNullable<string | null>;     // string
type J  = ReturnType<typeof fetchUser>;   // whatever fetchUser returns
type K  = Parameters<typeof fetchUser>;   // tuple of its parameter types
type L  = Awaited<Promise<User>>;         // User (recursively unwraps)
type M  = ConstructorParameters<typeof Error>;
type N  = InstanceType<typeof Error>;     // Error
type O  = Uppercase<"abc">;               // "ABC"
type P  = NoInfer<string>;                // blocks inference at this position (TS 5.
```

Composing them

TYPESCRIPT

```
// The input to a create endpoint: no server-generated fields, no id
type CreateUserInput = Omit<User, "id" | "createdAt">;

// The input to an update: everything optional except the id
type UpdateUserInput = Partial<Omit<User, "id" | "createdAt">> & Pick<User, "id">;

// The shape your API returns — dates become strings over JSON
type Serialized<T> = {
  [K in keyof T]: T[K] extends Date ? string
                : T[K] extends object ? Serialized<T[K]>
                : T[K]
};
type UserResponse = Serialized<User>;      // createdAt: string

// Awaited + ReturnType: the resolved type of an async function
async function fetchUser(id: string): Promise<User> { /* ... */ }
type FetchedUser = Awaited<ReturnType<typeof fetchUser>>;   // User
```

> ⚠ **`Omit` does not check its keys**
>
> `Omit<User, "emial">` compiles happily and silently omits nothing, because `Omit`'s second parameter is `keyof any`, not `keyof T`. A typo in an `Omit` is invisible. Define a strict version and use it instead — this catches real bugs during refactors where a field was renamed.

TYPESCRIPT

```
type StrictOmit<T, K extends keyof T> = Omit<T, K>;
// type Bad = StrictOmit<User, "emial">;      // ERROR — good

// Similarly, Exclude silently does nothing if U does not overlap T
type StrictExclude<T, U extends T> = Exclude<T, U>;
```

Utility types worth writing yourself

TYPESCRIPT

```ts
// 1. Prettify - forces the editor to show the resolved shape instead of
//    "Omit<User, 'id'> & Partial<...>". Enormously improves error messages.
type Prettify<T> = { [K in keyof T]: T[K] } & {};

// 2. At least one of these keys must be present
type RequireAtLeastOne<T, K extends keyof T = keyof T> =
  Omit<T, K> & { [P in K]-?: Required<Pick<T, P>> & Partial<Omit<T, P>> }[K];

type Contact = RequireAtLeastOne<{ email?: string; phone?: string; name: string }>;
const ok1: Contact = { name: "A", email: "a@b.com" };    // OK
// const bad: Contact = { name: "A" };                   // ERROR - good

// 3. Exactly one of these keys (a poor man's tagged union)
type ExactlyOne<T, K extends keyof T = keyof T> =
  { [P in K]: Required<Pick<T, P>> & Partial<Record<Exclude<K, P>, never>> }[K];

// 4. Make specific keys optional / required
type PartialBy<T, K extends keyof T>  = Omit<T, K> & Partial<Pick<T, K>>;
type RequiredBy<T, K extends keyof T> = Omit<T, K> & Required<Pick<T, K>>;

// 5. Union to intersection (used inside many library types)
type UnionToIntersection<U> =
  (U extends unknown ? (x: U) => void : never) extends (x: infer I) => void ? I : neve

// 6. Deep readonly / deep partial (defined in the previous section)

// 7. Opaque/branded id, as used earlier
type Brand<T, B extends string> = T & { readonly __brand: B };

// 8. Entries of an object, typed
type Entries<T> = { [K in keyof T]: [K, T[K]] }[keyof T][];
function entries<T extends object>(obj: T): Entries<T> {
  return Object.entries(obj) as Entries<T>;
}
```

✦ **`Prettify` is the single most useful helper you can add**

TypeScript error messages and hover tooltips often show a tangle like `Omit<User, "id" | "createdAt"> & Partial<Pick<User, "age">>`. Wrapping the type in `Prettify< ... >` forces it to resolve to a flat object literal in the tooltip. Put it in your shared types file on day one; your whole team's debugging gets faster.

`keyof`, `typeof` and indexed access

TYPESCRIPT

```typescript
const config = {
  host: "localhost",
  port: 5432,
  features: { auth: true, billing: false },
} as const;

type Config = typeof config;                        // the inferred shape
type ConfigKey = keyof Config;                      // "host" | "port" | "features"
type Port = Config["port"];                         // 5432
type FeatureKey = keyof Config["features"];         // "auth" | "billing"
type AnyValue = Config[keyof Config];               // "localhost" | 5432 | { ... }

// Indexed access on arrays
const roles = ["admin", "editor"] as const;
type Role = (typeof roles)[number];                 // "admin" | "editor"

// typeof on a function gives its signature
declare function handler(req: Request, res: Response): void;
type Handler = typeof handler;                      // (req: Request, res: Response) =>

// typeof on a class gives the CONSTRUCTOR type, not the instance type
class Service {}
type ServiceClass = typeof Service;                 // new () => Service
type ServiceInstance = Service;                     // the instance
type Also = InstanceType<typeof Service>;           // Service
```

Modules, Declaration Files and Typing Third-Party JS

At a customer you will constantly meet JavaScript libraries with no types, internal packages with broken types, and global objects injected by some script tag in 2017. Knowing how declaration files work turns each of these from a blocker into a ten-minute fix.

Import and export syntax that matters

TYPESCRIPT

```ts
// Value imports
import { createUser, type User } from "./users.js";      // inline type modifier
import type { Order, LineItem } from "./orders.js";      // type-only import
import defaultExport, { named } from "./mixed.js";
import * as utils from "./utils.js";

// Type-only exports - required under isolatedModules when re-exporting types
export type { User, Order };
export { createUser };
export * from "./shared.js";
export * as validators from "./validators.js";

// With verbatimModuleSyntax, `import type` is ERASED entirely and
// `import` is emitted verbatim. That makes side-effect imports explicit:
import "./polyfills.js";                                  // kept, runs for effect
```

⚠ **Why `import type` is not just style**

Under `isolatedModules` — which every esbuild/SWC/Vite setup uses — a transpiler sees one file at a time and cannot tell whether `import { Foo }` refers to a type or a value. If it keeps the import and `Foo` was a type, the emitted JS imports a non-existent binding and crashes at runtime. If it drops it and `Foo` was a value, you lose the import. `import type` removes the ambiguity. Turn on `verbatimModuleSyntax` and the compiler will tell you where you need it.

Declaration files

TYPESCRIPT

```typescript
// types/legacy-chart.d.ts
// Describing an untyped library you must use.

declare module "legacy-chart" {
  export interface ChartOptions {
    width?: number;
    height?: number;
    type: "line" | "bar" | "pie";
    data: Array<{ label: string; value: number }>;
    onClick?: (label: string, value: number) => void;
  }

  export class Chart {
    constructor(element: HTMLElement, options: ChartOptions);
    update(data: ChartOptions["data"]): void;
    destroy(): void;
  }

  export default function createChart(
    selector: string,
    options: ChartOptions
  ): Chart;
}
```

TYPESCRIPT

```ts
// types/globals.d.ts
// Ambient declarations: things that exist at runtime but have no import.

// A script-tag global
declare const APP_VERSION: string;
declare function trackEvent(name: string, props?: Record<string, unknown>): void;

// Augment the global Window — this REQUIRES interface declaration merging
declare global {
  interface Window {
    dataLayer: Array<Record<string, unknown>>;
    __APP_CONFIG__: { apiUrl: string; env: "dev" | "prod" };
  }

  // Typed environment variables — a genuinely high-value five minutes
  namespace NodeJS {
    interface ProcessEnv {
      NODE_ENV: "development" | "production" | "test";
      DATABASE_URL: string;
      API_KEY: string;
      PORT?: string;
    }
  }
}

// Non-code imports your bundler handles
declare module "*.svg" {
  const content: string;
  export default content;
}
declare module "*.css";
declare module "*.json" {
  const value: unknown;
  export default value;
}

export {};    // makes this file a MODULE; required for `declare global` to work
```

The `export {}` that makes it work

A `.d.ts` file with no top-level import or export is treated as a *script*, and its declarations go straight into the global scope — in which case `declare global` is both unnecessary and an error. Add `export {}` to make it a module, and then `declare global` is both valid and required. Getting this backwards produces the error *"Augmentations for the global scope can only be directly nested in external modules"*, which is one of TypeScript's less helpful messages.

Module augmentation

TYPESCRIPT

```ts
// Adding a property to an existing library's types - e.g. Express's Request
import "express";

declare module "express-serve-static-core" {
  interface Request {
    user?: { id: string; email: string; roles: string[] };
    requestId: string;
  }
}

// Now this type-checks everywhere, with no casts:
app.get("/me", (req, res) => {
  if (!req.user) return res.status(401).end();
  res.json({ id: req.user.id, requestId: req.requestId });
});
```

DefinitelyTyped and bundled types

Situation	What to do
Library ships its own types (`"types"` in package.json)	Nothing. It just works.
Types exist on DefinitelyTyped	`npm i -D @types/<name>`
No types anywhere, small surface	Write a minimal `.d.ts` covering only what you use
No types, huge surface, low risk	`declare module "lib";` — the whole module becomes `any`. Pragmatic, mark with a TODO.
Bundled types are wrong or outdated	Module augmentation, or a `paths` override pointing at your own `.d.ts`
Types conflict between two `@types` packages	Usually a version mismatch (classic with `@types/react`); pin with an `overrides` / `resolutions` field

BASH

```
# Find out whether types exist before you write any
npm view some-lib types typings
npm view @types/some-lib version

# Check what the installed types actually say
cat node_modules/some-lib/dist/index.d.ts

# When versions clash (very common with React 18 vs 19 types)
# package.json:
#   "overrides": { "@types/react": "18.3.12" }     (npm)
#   "resolutions": { "@types/react": "18.3.12" }   (yarn)
```

Publishing types for a library

JSON

```
{
  "name": "@customer/shared",
  "version": "1.0.0",
  "type": "module",
  "main": "./dist/index.cjs",
  "module": "./dist/index.js",
  "types": "./dist/index.d.ts",
  "exports": {
    ".": {
      "types": "./dist/index.d.ts",
      "import": "./dist/index.js",
      "require": "./dist/index.cjs"
    },
    "./schemas": {
      "types": "./dist/schemas.d.ts",
      "import": "./dist/schemas.js"
    }
  },
  "files": ["dist"],
  "scripts": {
    "build": "tsup src/index.ts src/schemas.ts --format esm,cjs --dts --clean",
    "check-exports": "attw --pack ."
  }
}
```

✦ **Put `"types"` first in every exports condition**

Export conditions are matched in declaration order, so if `"import"` comes before `"types"`, some resolvers never reach the types and your consumers get `any`. Run `attw` (`@arethetypeswrong/cli`) against your package before publishing — it catches this and about eight other packaging mistakes that are otherwise only discovered by a frustrated consumer.

Async Code: Promises, fetch and Error Typing

Asynchronous code is where TypeScript's limits are most visible: it can track what a promise resolves to, but it knows nothing about what it rejects with. Understanding that asymmetry — and designing around it — is most of what you need.

Promise basics

TYPESCRIPT

```typescript
// An async function always returns a Promise; the annotation is the RESOLVED type
async function getUser(id: string): Promise<User> {
  const res = await fetch(`/api/users/${id}`);
  return res.json() as Promise<User>;      // a LIE - see the next section
}

// Awaited unwraps nesting
type R = Awaited<Promise<Promise<User>>>;         // User

// Promise.all preserves the tuple positions - this is genuinely clever typing
const [user, orders, settings] = await Promise.all([
  getUser("1"),                       // Promise<User>
  getOrders("1"),                     // Promise<Order[]>
  getSettings("1"),                   // Promise<Settings>
]);                                   // [User, Order[], Settings]

// Promise.allSettled gives you a discriminated union per result
const results = await Promise.allSettled([getUser("1"), getUser("2")]);
for (const r of results) {
  if (r.status === "fulfilled") console.log(r.value.name);   // narrowed to User
  else console.error(r.reason);                              // unknown
}

// Typing a concurrency-limited map
async function mapLimit<T, R>(
  items: readonly T[],
  limit: number,
  fn: (item: T, index: number) => Promise<R>
): Promise<R[]> {
  const out: R[] = new Array(items.length);
  let cursor = 0;
  const workers = Array.from({ length: Math.min(limit, items.length) }, async () => {
    while (cursor < items.length) {
      const i = cursor++;
      out[i] = await fn(items[i]!, i);
    }
  });
  await Promise.all(workers);
  return out;
}
```

Error typing: the honest situation

TYPESCRIPT

```ts
// catch gives you `unknown` under strict mode (useUnknownInCatchVariables).
// This is correct: JavaScript can throw anything — a string, a number, null.
try {
  await risky();
} catch (e) {
  // e.message;                        // ERROR — e is unknown
  if (e instanceof Error) {
    console.error(e.message, e.stack);
  } else {
    console.error("Non-Error thrown:", e);
  }
}

// A reusable normaliser — put this in every codebase
function toError(e: unknown): Error {
  if (e instanceof Error) return e;
  if (typeof e === "string") return new Error(e);
  try { return new Error(JSON.stringify(e)); }
  catch { return new Error(String(e)); }
}

// Typed error hierarchy with discriminants so you can switch on them
abstract class AppError extends Error {
  abstract readonly code: string;
  abstract readonly httpStatus: number;
  constructor(message: string, readonly cause?: unknown) {
    super(message);
    this.name = new.target.name;         // so stack traces say "NotFoundError"
  }
}

class NotFoundError extends AppError {
  readonly code = "not_found" as const;
  readonly httpStatus = 404;
  constructor(readonly resource: string, readonly id: string) {
    super(`${resource} ${id} not found`);
  }
}

class ValidationError extends AppError {
  readonly code = "validation_failed" as const;
  readonly httpStatus = 422;
  constructor(readonly issues: Array<{ path: string; message: string }>) {
    super(`Validation failed: ${issues.length} issue(s)`);
  }
}

class UpstreamError extends AppError {
```

```
  readonly code = "upstream_failed" as const;
  readonly httpStatus = 502;
  constructor(readonly service: string, cause: unknown) {
    super(`Upstream ${service} failed`, cause);
  }
}

function toHttpResponse(e: unknown) {
  if (e instanceof AppError) {
    switch (e.code) {
      case "not_found":         return { status: 404, body: { error: e.code, resource:
      case "validation_failed": return { status: 422, body: { error: e.code, issues: e
      case "upstream_failed":   return { status: 502, body: { error: e.code, service:
    }
  }
  return { status: 500, body: { error: "internal" } };
}
```

⚠ **There is no `throws` clause and there never will be**

TypeScript deliberately does not track thrown types, because JavaScript's `throw` is unconstrained and checked exceptions have a poor track record. The practical consequences: you cannot know from a signature what a function might throw; the compiler will never remind you to handle a failure; and a refactor that introduces a new throw is invisible. If a failure is part of your domain — not found, invalid input, business rule violated — model it in the return type with a `Result`, and reserve `throw` for genuinely exceptional conditions handled by one top-level boundary.

Typed fetch and the cast that lies

TYPESCRIPT

```typescript
// WRONG — extremely common, and it is a runtime bug waiting to happen.
// res.json() returns Promise<any>. The cast asserts a shape nobody checked.
async function getUserBad(id: string): Promise<User> {
  const res = await fetch(`/api/users/${id}`);
  return (await res.json()) as User;
}
// If the API returns { id: 1 } instead of { id: "1" }, or omits `name`,
// nothing fails here. It fails three function calls away, in a component,
// with "Cannot read properties of undefined (reading 'toUpperCase')".

// RIGHT — validate at the boundary, then the types are TRUE for the rest of the progr
import { z } from "zod";

const UserSchema = z.object({
  id: z.string(),
  name: z.string().min(1),
  email: z.string().email(),
  age: z.number().int().positive().optional(),
  createdAt: z.coerce.date(),                // parses the ISO string into a Date
});
type User = z.infer<typeof UserSchema>;      // the type is DERIVED from the schema

async function getUser(id: string): Promise<User> {
  const res = await fetch(`/api/users/${encodeURIComponent(id)}`);
  if (!res.ok) throw new UpstreamError("users-api", `HTTP ${res.status}`);
  const json: unknown = await res.json();
  return UserSchema.parse(json);             // throws ZodError with a precise path
}
```

AbortController and timeouts

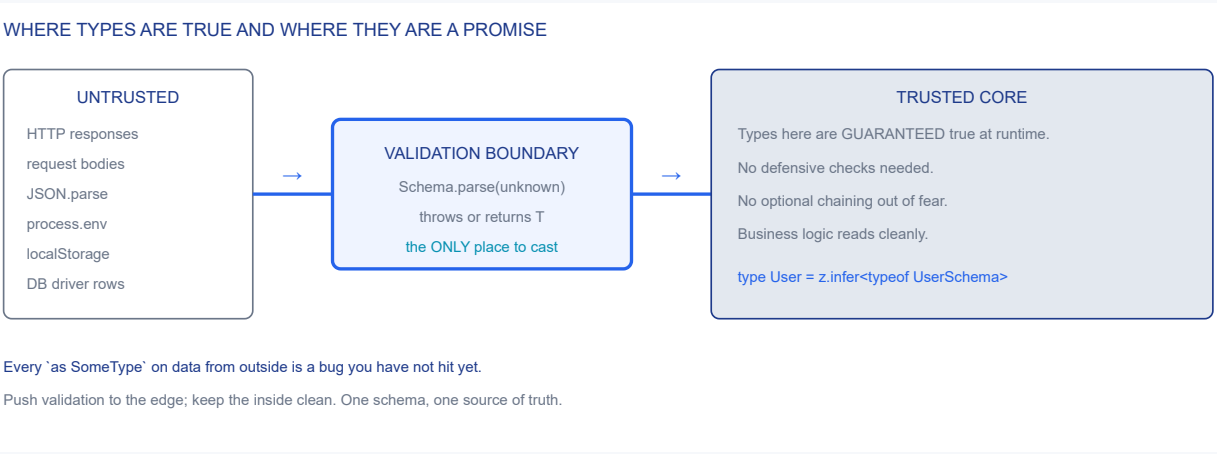
TYPESCRIPT

```ts
async function fetchWithTimeout(
  url: string,
  init: RequestInit = {},
  timeoutMs = 10_000
): Promise<Response> {
  const controller = new AbortController();
  const timer = setTimeout(() => controller.abort(new Error("timeout")), timeoutMs);
  try {
    return await fetch(url, { ...init, signal: controller.signal });
  } catch (e) {
    if (e instanceof DOMException && e.name === "AbortError") {
      throw new UpstreamError(url, `timed out after ${timeoutMs}ms`);
    }
    throw e;
  } finally {
    clearTimeout(timer);
  }
}

// Composing an external signal (e.g. React cleanup) with a timeout
function withTimeout(signal: AbortSignal | undefined, ms: number): AbortSignal {
  const signals = [AbortSignal.timeout(ms)];
  if (signal) signals.push(signal);
  return AbortSignal.any(signals);
}
```

Runtime Validation with Zod

Types are a compile-time fiction. Validation is the runtime reality. The crucial insight is that you should write the **schema** and derive the **type** from it — never the other way round — so that the thing you check and the thing you declare can never diverge. This section is the most practically important one in the guide.



Validate once at the edge. Everything inside can then trust its types completely.

Zod essentials

TYPESCRIPT

```ts
import { z } from "zod";

const AddressSchema = z.object({
  line1: z.string().min(1, "Address line 1 is required"),
  line2: z.string().optional(),
  city: z.string().min(1),
  postcode: z.string().regex(/^\d{6}$/, "Must be a 6-digit PIN code"),
  country: z.string().length(2).default("IN"),
});

const OrderSchema = z.object({
  id: z.string().uuid(),
  customerId: z.string().uuid(),
  status: z.enum(["draft", "paid", "shipped", "cancelled"]),
  lines: z.array(z.object({
    sku: z.string(),
    quantity: z.number().int().positive(),
    unitPriceMinor: z.number().int().nonnegative(),   // store money as integer minor
  })).min(1, "An order needs at least one line"),
  shipping: AddressSchema,
  placedAt: z.coerce.date(),
  notes: z.string().max(500).nullable(),
  metadata: z.record(z.string(), z.string()).default({}),
})
.refine(
  (o) => o.status !== "shipped" || o.lines.length > 0,
  { message: "A shipped order must have lines", path: ["lines"] }
);

type Order = z.infer<typeof OrderSchema>;     // THE type. Never hand-write it.
type OrderInput = z.input<typeof OrderSchema>; // before defaults/coercion
type OrderOutput = z.output<typeof OrderSchema>; // after - same as z.infer
```

parse vs safeParse

TYPESCRIPT

```typescript
// parse: throws ZodError on failure. Use where a throw is handled upstream.
const order = OrderSchema.parse(json);        // Order, guaranteed

// safeParse: returns a discriminated union. Use where you must handle it inline.
const result = OrderSchema.safeParse(json);
if (!result.success) {
  // result.error.issues is an array with path, code and message per problem
  const problems = result.error.issues.map((i) => ({
    field: i.path.join("."),
    message: i.message,
    code: i.code,
  }));
  return { status: 422, body: { error: "validation_failed", problems } };
}
const order2 = result.data;                   // Order, narrowed

// Formatting for a form UI
const flat = result.success ? null : z.flattenError(result.error);
// { formErrors: string[], fieldErrors: { [k: string]: string[] } }
```

Discriminated unions in Zod

TYPESCRIPT

```
const EventSchema = z.discriminatedUnion("type", [
  z.object({
    type: z.literal("order.created"),
    orderId: z.string().uuid(),
    totalMinor: z.number().int(),
  }),
  z.object({
    type: z.literal("order.paid"),
    orderId: z.string().uuid(),
    paymentId: z.string(),
    paidAt: z.coerce.date(),
  }),
  z.object({
    type: z.literal("order.cancelled"),
    orderId: z.string().uuid(),
    reason: z.string().min(1),
  }),
]);

type DomainEvent = z.infer<typeof EventSchema>;

// The inferred type IS a discriminated union, so switch narrowing works perfectly.
function handle(raw: unknown) {
  const event = EventSchema.parse(raw);
  switch (event.type) {
    case "order.created":   return onCreated(event.totalMinor);
    case "order.paid":      return onPaid(event.paymentId, event.paidAt);
    case "order.cancelled": return onCancelled(event.reason);
    default: {
      const _never: never = event;
      throw new Error(`Unhandled event: ${JSON.stringify(_never)}`);
    }
  }
}
```

Validating environment variables

TYPESCRIPT

```ts
// src/env.ts — import this ONCE at startup. Fails fast with a readable message.
import { z } from "zod";

const EnvSchema = z.object({
  NODE_ENV: z.enum(["development", "test", "production"]).default("development"),
  PORT: z.coerce.number().int().min(1).max(65535).default(3000),
  DATABASE_URL: z.string().url(),
  REDIS_URL: z.string().url().optional(),
  JWT_SECRET: z.string().min(32, "JWT_SECRET must be at least 32 characters"),
  LOG_LEVEL: z.enum(["debug", "info", "warn", "error"]).default("info"),
  FEATURE_FLAGS: z.string()
    .transform((s) => new Set(s.split(",").map((f) => f.trim()).filter(Boolean)))
    .default(""),
});

const parsed = EnvSchema.safeParse(process.env);
if (!parsed.success) {
  console.error("Invalid environment configuration:");
  for (const issue of parsed.error.issues) {
    console.error(`  ${issue.path.join(".")}: ${issue.message}`);
  }
  process.exit(1);
}

export const env = parsed.data;
export type Env = typeof env;
```

✓ **This is a twenty-minute change with outsized value**

Every team has been woken at 3 a.m. by a service that started fine and then failed on the first request because `DATABASE_URL` was missing or `PORT` was the string `"3000"`. Validating the environment at startup converts that into a container that refuses to start with a message naming the exact variable. On a customer engagement this is one of those small things that visibly improves their platform and takes no negotiation to introduce.

An end-to-end typed API client

TYPESCRIPT

```ts
// src/api/client.ts
import { z } from "zod";

export class ApiError extends Error {
  constructor(
    readonly status: number,
    readonly url: string,
    readonly body: unknown,
  ) {
    super(`API ${status} on ${url}`);
    this.name = "ApiError";
  }
}

export class ApiValidationError extends Error {
  constructor(readonly url: string, readonly issues: z.ZodIssue[]) {
    super(`Response from ${url} did not match the expected schema`);
    this.name = "ApiValidationError";
  }
}

interface RequestOptions<TBody> {
  method?: "GET" | "POST" | "PUT" | "PATCH" | "DELETE";
  body?: TBody;
  query?: Record<string, string | number | boolean | undefined>;
  signal?: AbortSignal;
  timeoutMs?: number;
}

export class ApiClient {
  constructor(
    private readonly baseUrl: string,
    private readonly getToken: () => string | null = () => null,
  ) {}

  async request<TResponse, TBody = never>(
    path: string,
    schema: z.ZodType<TResponse>,
    options: RequestOptions<TBody> = {},
  ): Promise<TResponse> {
    const { method = "GET", body, query, signal, timeoutMs = 15_000 } = options;

    const url = new URL(path, this.baseUrl);
    for (const [k, v] of Object.entries(query ?? {})) {
      if (v !== undefined) url.searchParams.set(k, String(v));
    }

    const token = this.getToken();
```

```
    const headers: Record<string, string> = { Accept: "application/json" };
    if (body !== undefined) headers["Content-Type"] = "application/json";
    if (token) headers.Authorization = `Bearer ${token}`;

    const signals = [AbortSignal.timeout(timeoutMs)];
    if (signal) signals.push(signal);

    const res = await fetch(url, {
      method,
      headers,
      body: body === undefined ? undefined : JSON.stringify(body),
      signal: AbortSignal.any(signals),
    });

    const text = await res.text();
    const json: unknown = text ? JSON.parse(text) : null;

    if (!res.ok) throw new ApiError(res.status, url.pathname, json);

    const parsed = schema.safeParse(json);
    if (!parsed.success) {
      // Log the full issue list - this is how you find API drift in minutes
      console.error("Schema mismatch", url.pathname, parsed.error.issues);
      throw new ApiValidationError(url.pathname, parsed.error.issues);
    }
    return parsed.data;
  }
}
```

TYPESCRIPT

```ts
// src/api/users.ts — one module per resource. Schema and function together.
import { z } from "zod";
import { ApiClient } from "./client.js";

export const UserSchema = z.object({
  id: z.string().uuid(),
  name: z.string(),
  email: z.string().email(),
  role: z.enum(["admin", "editor", "viewer"]),
  createdAt: z.coerce.date(),
});
export type User = z.infer<typeof UserSchema>;

export const PageSchema = <T extends z.ZodTypeAny>(item: T) =>
  z.object({
    items: z.array(item),
    total: z.number().int().nonnegative(),
    page: z.number().int().positive(),
    pageSize: z.number().int().positive(),
  });

export const CreateUserSchema = UserSchema.omit({ id: true, createdAt: true });
export type CreateUserInput = z.infer<typeof CreateUserSchema>;

export function usersApi(client: ApiClient) {
  return {
    get: (id: string) =>
      client.request(`/users/${encodeURIComponent(id)}`, UserSchema),

    list: (page = 1, pageSize = 25) =>
      client.request("/users", PageSchema(UserSchema), { query: { page, pageSize } }),

    create: (input: CreateUserInput) =>
      client.request("/users", UserSchema, {
        method: "POST",
        body: CreateUserSchema.parse(input),      // validate OUTBOUND too
      }),

    remove: (id: string) =>
      client.request(`/users/${encodeURIComponent(id)}`, z.object({ deleted: z.literal
        method: "DELETE",
      }),
  };
}

// Usage — fully typed, and TRUE at runtime
const api = usersApi(new ApiClient("https://api.customer.com/v1", () => session.token)
```

```
const page = await api.list(1, 50);
page.items[0]?.createdAt.getFullYear();          // a real Date, because of z.coerce.d
```

✦ **Validating responses catches API drift the day it happens**
When the customer's backend team renames a field or changes a type, your client throws a precise error naming the field, in staging, on the first request — instead of producing `undefined` that surfaces as a blank screen three screens later. The small cost is a schema per endpoint; the payoff is that integration bugs become self-diagnosing. If performance matters on a hot path, validate in non-production environments only via an env flag.

Zod alternatives

Library	Strength	Trade-off
Zod	Best ergonomics, huge ecosystem, excellent inference, great error messages	Larger bundle; slower than compiled validators
Valibot	Modular, tree-shakes to a fraction of Zod's size	Smaller ecosystem; different API shape
ArkType	Very fast, type-syntax-like definitions	Newer; fewer integrations
TypeBox	Produces real JSON Schema — ideal if you also need OpenAPI	More verbose
io-ts	Functional, composable, fp-ts ecosystem	Steep learning curve for most teams

Typing React Properly

React is where most TypeScript is written, and where most of it is written badly — `React.FC` everywhere, `any` on every event handler, props typed as `object`. The correct patterns are not hard and they make component APIs self-documenting, which matters enormously when you are handing a codebase to a customer's team.

Props

TYPESCRIPT

```tsx
// Plain function with a typed props object. Do NOT use React.FC.
interface ButtonProps {
  label: string;
  onClick: () => void;
  variant?: "primary" | "secondary" | "danger";
  disabled?: boolean;
  children?: React.ReactNode;
}

function Button({ label, onClick, variant = "primary", disabled = false }: ButtonProps
  return (
    <button className={`btn btn-${variant}`} onClick={onClick} disabled={disabled}>
      {label}
    </button>
  );
}

// Extending native element props - the pattern for any wrapper component
interface InputProps extends React.ComponentPropsWithoutRef<"input"> {
  label: string;
  error?: string;
}

function Input({ label, error, id, ...rest }: InputProps) {
  const inputId = id ?? React.useId();
  return (
    <div>
      <label htmlFor={inputId}>{label}</label>
      <input id={inputId} aria-invalid={!!error} {...rest} />
      {error && <span role="alert">{error}</span>}
    </div>
  );
}

// Forwarding a ref with the right element type
const FancyInput = React.forwardRef<HTMLInputElement, InputProps>(
  function FancyInput({ label, error, ...rest }, ref) {
    return <input ref={ref} aria-label={label} {...rest} />;
  }
);

// React 19: ref is just another prop, no forwardRef needed
interface Input19Props extends React.ComponentPropsWithRef<"input"> {
  label: string;
}
function Input19({ label, ref, ...rest }: Input19Props) {
```

```
  return <input ref={ref} aria-label={label} {...rest} />;
}
```

> ⚠ **Why not `React.FC`**
>
> Historically `React.FC` implicitly added `children` to every component, which meant a component that should not accept children silently did. React 18's types removed the implicit children, which was a breaking change for everyone relying on it. It also complicates generic components, since you cannot easily write a generic `React.FC`. A plain function with a typed props parameter has none of these problems, infers the return type correctly, and is what the React team recommends. Declare `children: React.ReactNode` explicitly when you want it.

Discriminated union props

TYPESCRIPT

```
// Express mutually exclusive prop combinations in the type, not in a runtime check.
type AlertProps =
  | { severity: "info" | "success"; message: string }
  | { severity: "error"; message: string; onRetry: () => void; errorCode: string };

function Alert(props: AlertProps) {
  return (
    <div role="alert" className={`alert alert-${props.severity}`}>
      {props.message}
      {props.severity === "error" && (
        <button onClick={props.onRetry}>Retry ({props.errorCode})</button>
      )}
    </div>
  );
}

// <Alert severity="error" message="Boom" />    // ERROR: onRetry and errorCode require
// <Alert severity="info" message="Hi" onRetry={fn} />  // ERROR: onRetry not allowed
```

Hooks

TYPESCRIPT

```tsx
// useState: inference is usually right; annotate when the initial value is narrower
const [count, setCount] = React.useState(0);                       // number
const [user, setUser] = React.useState<User | null>(null);         // needs the annotat
const [items, setItems] = React.useState<string[]>([]);            // otherwise never[]

// useReducer: type the state and the action union
type State = { count: number; step: number };
type Action =
  | { type: "increment" }
  | { type: "decrement" }
  | { type: "setStep"; step: number }
  | { type: "reset" };

function reducer(state: State, action: Action): State {
  switch (action.type) {
    case "increment": return { ...state, count: state.count + state.step };
    case "decrement": return { ...state, count: state.count - state.step };
    case "setStep":   return { ...state, step: action.step };
    case "reset":     return { count: 0, step: 1 };
    default: {
      const _never: never = action;
      return state;
    }
  }
}
const [state, dispatch] = React.useReducer(reducer, { count: 0, step: 1 });

// useRef: three distinct forms, and people get them confused
const inputRef = React.useRef<HTMLInputElement>(null);     // for a DOM ref (readonly
const timerRef = React.useRef<number | undefined>(undefined); // mutable box
const latest = React.useRef(props.value);                  // inferred from initial

// useCallback and useMemo infer correctly from the function you pass
const handleSelect = React.useCallback((id: string) => setSelected(id), []);
const total = React.useMemo(() => items.reduce((a, b) => a + b.price, 0), [items]);

// A custom hook: return `as const` so the tuple keeps its positions
function useToggle(initial = false) {
  const [on, setOn] = React.useState(initial);
  const toggle = React.useCallback(() => setOn((v) => !v), []);
  return [on, toggle, setOn] as const;      // [boolean, () => void, Dispatch< ... >]
}
```

Events

TYPESCRIPT

```tsx
function Form() {
  // Annotate the event with the element it comes from
  const onSubmit = (e: React.FormEvent<HTMLFormElement>) => {
    e.preventDefault();
    const data = new FormData(e.currentTarget);      // currentTarget is typed
    const name = data.get("name");                   // FormDataEntryValue | null
  };

  const onChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    setValue(e.target.value);                        // string
  };

  const onClick = (e: React.MouseEvent<HTMLButtonElement>) => { /* ... */ };
  const onKeyDown = (e: React.KeyboardEvent<HTMLInputElement>) => {
    if (e.key === "Enter" && !e.shiftKey) submit();
  };

  // Inline handlers are contextually typed — no annotation needed
  return <input onChange={(e) => setValue(e.target.value)} />;
}
```

✦ **`target` vs `currentTarget`**

`currentTarget` is the element the handler is attached to and is precisely typed. `target` is whatever was actually clicked, which could be a child, so React types it as `EventTarget` for most events. For `ChangeEvent` React specially types `target` as the element, which is why `e.target.value` works there and not on `MouseEvent`. When you need the form in a submit handler, always use `currentTarget`.

Generic components

TYPESCRIPT

```tsx
interface SelectProps<T> {
  items: readonly T[];
  value: T | null;
  onChange: (value: T) => void;
  getKey: (item: T) => string;
  renderItem: (item: T) => React.ReactNode;
  placeholder?: string;
}

function Select<T>({ items, value, onChange, getKey, renderItem, placeholder }: Select
  return (
    <ul role="listbox">
      {placeholder && !value && <li className="placeholder">{placeholder}</li>}
      {items.map((item) => (
        <li
          key={getKey(item)}
          role="option"
          aria-selected={value !== null && getKey(value) === getKey(item)}
          onClick={() => onChange(item)}
        >
          {renderItem(item)}
        </li>
      ))}
    </ul>
  );
}

// T is inferred from `items`; onChange receives a User, checked.
<Select
  items={users}
  value={selectedUser}
  onChange={setSelectedUser}
  getKey={(u) => u.id}
  renderItem={(u) => <span>{u.name}</span>}
/>;

// A generic table with typed column accessors
interface Column<T> {
  header: string;
  accessor: keyof T | ((row: T) => React.ReactNode);
  width?: string;
}

function DataTable<T extends { id: string }>({ rows, columns }: {
  rows: readonly T[];
  columns: ReadonlyArray<Column<T>>;
}) {
  return (
```

```
    <table>
      <thead><tr>{columns.map((c) => <th key={c.header}>{c.header}</th>)}</tr></thead>
      <tbody>
        {rows.map((row) => (
          <tr key={row.id}>
            {columns.map((c) => (
              <td key={c.header}>
                {typeof c.accessor === "function"
                  ? c.accessor(row)
                  : String(row[c.accessor])}
              </td>
            ))}
          </tr>
        ))}
      </tbody>
    </table>
  );
}
```

> ⚠ **Generic arrow components in `.tsx` files**
>
> `const Select = <T>(props: Props<T>) => ...` fails to parse in a `.tsx` file because `<T>` looks like a JSX tag. Workarounds: use a `function` declaration (cleanest), write `<T,>` with a trailing comma, or write `<T extends unknown>`. A plain function declaration avoids the question entirely and reads better.

Context

TYPESCRIPT

```tsx
interface AuthContextValue {
  user: User | null;
  login: (email: string, password: string) => Promise<void>;
  logout: () => void;
  isLoading: boolean;
}

// Default of null plus a hook that throws - removes `| null` from every consumer
const AuthContext = React.createContext<AuthContextValue | null>(null);

export function useAuth(): AuthContextValue {
  const ctx = React.useContext(AuthContext);
  if (ctx === null) {
    throw new Error("useAuth must be used inside <AuthProvider>");
  }
  return ctx;                                   // AuthContextValue, never null
}

export function AuthProvider({ children }: { children: React.ReactNode }) {
  const [user, setUser] = React.useState<User | null>(null);
  const [isLoading, setIsLoading] = React.useState(false);

  const value = React.useMemo<AuthContextValue>(() => ({
    user,
    isLoading,
    login: async (email, password) => {
      setIsLoading(true);
      try { setUser(await api.login(email, password)); }
      finally { setIsLoading(false); }
    },
    logout: () => setUser(null),
  }), [user, isLoading]);

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}
```

Typing a Node Backend

Server-side TypeScript is where the validation story matters most, because every input is hostile and every output is a contract. The patterns below apply whether the customer runs Express, Fastify, Hono or NestJS — the framework changes, the shape of the solution does not.

Express with validated handlers

TYPESCRIPT

```ts
import express, { type NextFunction, type Request, type Response } from "express";
import { z } from "zod";

// Augment Request once, in a .d.ts, so middleware-set properties are typed everywhere
declare module "express-serve-static-core" {
  interface Request {
    user?: { id: string; roles: readonly string[] };
    requestId: string;
  }
}

// A generic wrapper that validates params, query and body, then hands you typed data.
function validated<
  P extends z.ZodTypeAny,
  Q extends z.ZodTypeAny,
  B extends z.ZodTypeAny,
  R,
>(
  schemas: { params?: P; query?: Q; body?: B },
  handler: (
    input: { params: z.infer<P>; query: z.infer<Q>; body: z.infer<B> },
    req: Request,
    res: Response,
  ) => Promise<R>,
) {
  return async (req: Request, res: Response, next: NextFunction) => {
    try {
      const params = schemas.params ? schemas.params.parse(req.params) : undefined;
      const query  = schemas.query  ? schemas.query.parse(req.query)   : undefined;
      const body   = schemas.body   ? schemas.body.parse(req.body)     : undefined;
      const result = await handler({ params, query, body } as never, req, res);
      if (!res.headersSent) res.json(result);
    } catch (e) {
      next(e);                                  // one error handler, defined once
    }
  };
}

const app = express();
app.use(express.json({ limit: "1mb" }));

const CreateOrderBody = z.object({
  customerId: z.string().uuid(),
  lines: z.array(z.object({
    sku: z.string().min(1),
    quantity: z.number().int().positive().max(999),
  })).min(1),
  note: z.string().max(500).optional(),
```

```
});

const ListQuery = z.object({
  page: z.coerce.number().int().positive().default(1),
  pageSize: z.coerce.number().int().min(1).max(100).default(25),
  status: z.enum(["draft", "paid", "shipped", "cancelled"]).optional(),
});

app.post("/orders", validated({ body: CreateOrderBody }, async ({ body }, req) => {
  // body is fully typed AND validated. No casts, no optional chaining out of fear.
  return orderService.create(body, req.user?.id);
}));

app.get("/orders", validated({ query: ListQuery }, async ({ query }) => {
  return orderService.list(query);            // query.page is a number, defaulted
}));

// Single error handler - the only place that maps domain errors to HTTP
app.use((err: unknown, req: Request, res: Response, _next: NextFunction) => {
  if (err instanceof z.ZodError) {
    return res.status(422).json({
      error: "validation_failed",
      requestId: req.requestId,
      issues: err.issues.map((i) => ({ path: i.path.join("."), message: i.message })),
    });
  }
  if (err instanceof AppError) {
    return res.status(err.httpStatus).json({ error: err.code, message: err.message });
  }
  console.error({ requestId: req.requestId, err });
  res.status(500).json({ error: "internal", requestId: req.requestId });
});
```

A typed service and repository layer

TYPESCRIPT

```ts
// Domain types live in one place and are derived from schemas.
export const OrderSchema = z.object({
  id: z.string().uuid(),
  customerId: z.string().uuid(),
  status: z.enum(["draft", "paid", "shipped", "cancelled"]),
  totalMinor: z.number().int().nonnegative(),
  createdAt: z.coerce.date(),
});
export type Order = z.infer<typeof OrderSchema>;

// The repository interface is what the service depends on - not the driver.
export interface OrderRepository {
  findById(id: string): Promise<Order | null>;
  findByCustomer(customerId: string, page: number, size: number): Promise<Order[]>;
  insert(order: Omit<Order, "createdAt">): Promise<Order>;
  updateStatus(id: string, status: Order["status"]): Promise<Order>;
}

// Implementation validates rows coming OUT of the database too. Drivers return `any`.
export class PgOrderRepository implements OrderRepository {
  constructor(private readonly db: Pool) {}

  async findById(id: string): Promise<Order | null> {
    const { rows } = await this.db.query("select * from orders where id = $1", [id]);
    if (rows.length === 0) return null;
    return OrderSchema.parse(rows[0]);        // catches a migration you forgot about
  }

  async insert(order: Omit<Order, "createdAt">): Promise<Order> {
    const { rows } = await this.db.query(
      `insert into orders (id, customer_id, status, total_minor)
       values ($1, $2, $3, $4) returning *`,
      [order.id, order.customerId, order.status, order.totalMinor],
    );
    return OrderSchema.parse(rows[0]);
  }
  // ...
}

// The service is pure logic over the interface - trivially unit-testable with a fake.
export class OrderService {
  constructor(
    private readonly repo: OrderRepository,
    private readonly payments: PaymentGateway,
  ) {}

  async markPaid(id: string, paymentId: string): Promise<Order> {
    const order = await this.repo.findById(id);
```

```
    if (!order) throw new NotFoundError("Order", id);
    if (order.status !== "draft") {
      throw new ConflictError(`Order ${id} is ${order.status}, cannot be paid`);
    }
    await this.payments.capture(paymentId, order.totalMinor);
    return this.repo.updateStatus(id, "paid");
  }
}
```

> ⚠ **Database drivers return `any`, which poisons everything downstream**
>
> `pg`, `mysql2` and `mongodb` all hand you loosely typed rows. Writing `rows[0] as Order` is exactly the lie described earlier: a column renamed in a migration, a `numeric` that arrives as a string, a nullable column you forgot — none of them produce an error until something far away breaks. Parse rows through the schema, or use a query builder that generates types from the real schema (Drizzle, Kysely, Prisma). The parse costs microseconds and turns a class of silent corruption into a loud, located error.

Fastify and Hono: schema-first by design

TYPESCRIPT

```
// Fastify with a type provider infers handler types from the schema automatically.
import Fastify from "fastify";
import { type ZodTypeProvider, serializerCompiler, validatorCompiler } from "fastify-t

const app = Fastify().withTypeProvider<ZodTypeProvider>();
app.setValidatorCompiler(validatorCompiler);
app.setSerializerCompiler(serializerCompiler);

app.route({
  method: "POST",
  url: "/orders",
  schema: {
    body: CreateOrderBody,
    response: { 201: OrderSchema },
  },
  handler: async (req, reply) => {
    // req.body is typed from the schema, no manual parse call
    const order = await orderService.create(req.body);
    return reply.code(201).send(order);       // the response is validated too
  },
});
```

TYPESCRIPT

```typescript
// Hono with zod-validator — and the client gets types for free via hc<AppType>
import { Hono } from "hono";
import { zValidator } from "@hono/zod-validator";

const api = new Hono()
  .post("/orders", zValidator("json", CreateOrderBody), async (c) => {
    const body = c.req.valid("json");          // typed
    return c.json(await orderService.create(body), 201);
  })
  .get("/orders/:id", async (c) => {
    const order = await orderService.get(c.req.param("id"));
    return c.json(order);
  });

export type AppType = typeof api;

// In the frontend package:
// import { hc } from "hono/client";
// const client = hc<AppType>("https://api.customer.com");
// const res = await client.orders.$post({ json: { customerId, lines } });
// End-to-end types with no code generation step.
```

Errors You Will Hit and How to Read Them

TypeScript error messages are famously long, and the skill is knowing which part to read. This section covers the half-dozen errors that account for most of the confusion, plus a method for decoding a forty-line message in under a minute.

Reading a long error

Read the LAST line first, then the first

TypeScript errors are structured as a chain: "A is not assignable to B" at the top, then progressively more specific "because X is not assignable to Y" lines, ending with the actual mismatch. The final line names the concrete incompatibility — `Type 'string' is not assignable to type 'number'`. Read that, then look at the first line to find which argument or property it was about, and ignore the middle entirely. In the editor, hovering the squiggle and collapsing the message gives the same effect.

Excess property checking

TYPESCRIPT

```typescript
interface Options { width: number; height?: number }

function render(o: Options) {}

render({ width: 10, heigth: 20 });
// ERROR: Object literal may only specify known properties,
//        and 'heigth' does not exist in type 'Options'. Did you mean 'height'?

// But this compiles:
const opts = { width: 10, heigth: 20 };
render(opts);                                  // no error - structurally compatible

// Why: excess property checking applies ONLY to FRESH object literals assigned
// directly. Once the literal is in a variable, its "freshness" is gone and
// normal structural compatibility applies (extra props are allowed).

// If you want the check on a variable, use satisfies:
const opts2 = { width: 10, heigth: 20 } satisfies Options;   // ERROR - good
```

Variance: why callbacks behave oddly

TYPESCRIPT

```
// Arrays are covariant in TypeScript, which is UNSOUND but pragmatic.
class Animal { name = "" }
class Dog extends Animal { breed = "" }

const dogs: Dog[] = [new Dog()];
const animals: Animal[] = dogs;             // allowed
animals.push(new Animal());                 // compiles - but dogs now contains a non-
// dogs[1].breed;                           // undefined at runtime. TypeScript chose
//                                          // convenience over soundness here.

// readonly arrays avoid the problem
const safe: readonly Animal[] = dogs;       // cannot push

// Function PARAMETERS are bivariant for method syntax, contravariant for
// property syntax under strictFunctionTypes. This is the real-world difference:
interface Handler1 { handle(e: Animal): void }      // method - bivariant (lenient)
interface Handler2 { handle: (e: Animal) => void }  // property - contravariant (stric

const h1: Handler1 = { handle: (d: Dog) => {} };    // allowed (bivariance)
// const h2: Handler2 = { handle: (d: Dog) => {} }; // ERROR under strictFunctionTypes

// Rule of thumb: declare callbacks as properties, not methods, if you want
// the compiler to catch a handler that demands more than it will receive.
```

Type widening surprises

TYPESCRIPT

```
// 1. Object literal properties widen even inside a const
const config = { mode: "dark" };            // mode: string, NOT "dark"
setTheme(config);                           // ERROR if setTheme wants { mode: "dark"
const config2 = { mode: "dark" } as const;  // mode: "dark"

// 2. Arrays widen to the union element type
const pair = [1, "a"];                      // (string | number)[], not [number, strin
const tuple = [1, "a"] as const;            // readonly [1, "a"]

// 3. Return type of a function returning an object literal widens
function makeConfig() { return { mode: "dark" }; }   // { mode: string }
function makeConfig2() { return { mode: "dark" } as const; }   // { readonly mode: "da

// 4. let vs const
let a = "x";                                // string
const b = "x";                              // "x"
let c: "x" = "x";                           // "x" - annotate to keep it narrow
```

Declaration merging gotchas

TYPESCRIPT

```
// Merging works for interfaces, namespaces and enums - but silently.
interface Config { a: string }
interface Config { b: number }               // merged: { a: string; b: number }
// interface Config { a: number }            // ERROR - conflicting member

// A common confusion: you add a property to a library interface, it works
// locally, then fails in CI because the .d.ts file was not in `include`.
// Ambient augmentations must be inside tsconfig's include/files or typeRoots.

// Classes merge with namespaces (used for static-like members on a class)
class Album {}
namespace Album {
  export const DEFAULT_TRACKS = 10;
}
Album.DEFAULT_TRACKS;                        // 10
```

The errors you will actually see, decoded

Error	What it usually means	Fix
`Object is possibly 'undefined'`	Optional property, array index with `noUncheckedIndexedAccess`, or a map lookup	Guard, `??`, optional chaining, or a non-null assertion *only* when you can prove it
`Type 'X' is not assignable to type 'Y'` (long chain)	A nested property mismatch	Read the last line of the chain; it names the actual pair
`Property 'x' does not exist on type 'A | B'`	You are on a union and `x` is only on one member	Narrow with a discriminant first
`Argument of type 'unknown'…`	Data from `catch`, `JSON.parse` or an API	Validate or narrow — do not cast
`No overload matches this call`	Any argument is wrong, in an overloaded function	Check each argument individually; expand the "Overload 1 of N" detail
`Type instantiation is excessively deep`	Recursive type with no depth guard	Add a depth counter tuple, or simplify
`Cannot find module './x' or its type declarations`	Module resolution mismatch	Check `moduleResolution`; add `.js` under NodeNext
`Cannot redeclare block-scoped variable`	A global `.d.ts` script file colliding	Add `export {}` to make it a module
`This expression is not callable. Type 'never' has no call signatures`	Indexing a union of object types with a dynamic key	Narrow first, or cast the lookup at the single point of access
`Excessive stack depth comparing types`	Two large recursive types being compared	Introduce an interface boundary to break the recursion

Escape hatches, ranked by how much they should worry you

TYPESCRIPT

```ts
// 1. satisfies - no risk. Checks without widening.
const x = { a: 1 } satisfies Record<string, number>;

// 2. Type predicate / assertion function - risk is contained to one function
function isUser(v: unknown): v is User { return UserSchema.safeParse(v).success; }

// 3. Non-null assertion - you are claiming knowledge the compiler lacks
const el = document.getElementById("root")!;        // crashes if wrong
// Prefer: const el = document.getElementById("root"); if (!el) throw ...

// 4. `as` cast - the compiler stops checking this assignment
const u = raw as User;                              // a lie if raw is not a User

// 5. `as unknown as T` - double cast, defeats even the "sufficiently overlapping" che
const weird = thing as unknown as Completely Different;   // almost always wrong

// 6. `any` - contagious, disables checking downstream
let whatever: any = getThing();

// 7. @ts-ignore - suppresses the NEXT line, including errors you did not intend
// @ts-ignore
brokenThing.doStuff();

// 8. @ts-expect-error - better than ts-ignore: it ERRORS if the line stops failing,
//    so it cannot rot silently. Always prefer this, and always add a reason.
// @ts-expect-error library types are wrong, see https://github.com/x/y/issues/123
legacy.doStuff();
```

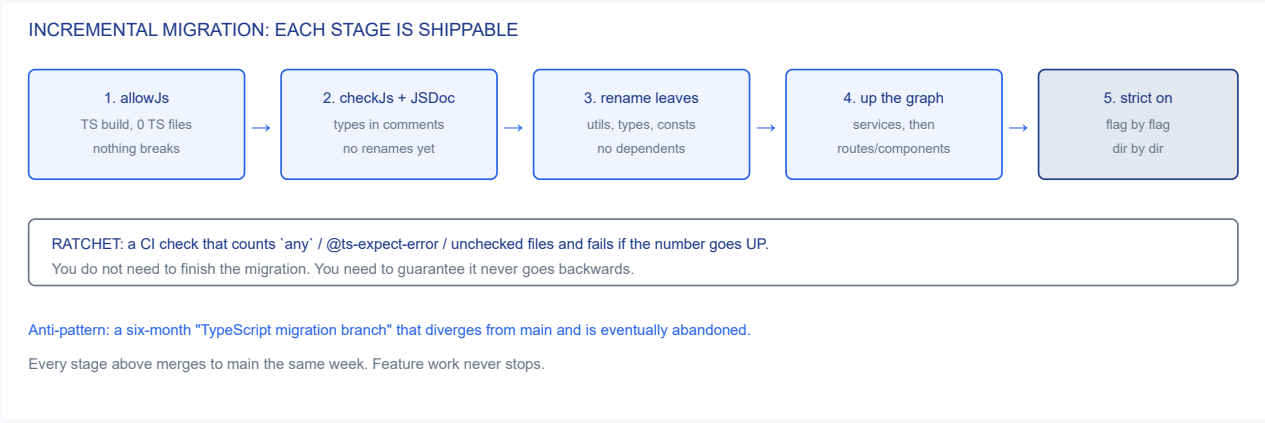
✓ **Use `@ts-expect-error`, never `@ts-ignore`**

They look equivalent but behave oppositely when the underlying problem is fixed. `@ts-ignore` keeps suppressing forever and nobody ever removes it. `@ts-expect-error` becomes an error itself once the line compiles cleanly — so when the library ships correct types, your build tells you to delete the comment. Enforce it with the ESLint rule `@typescript-eslint/ban-ts-comment` configured to allow `expect-error` with a description and ban the rest.

Migrating a JavaScript Codebase

You will be asked to do this at a customer, often with a team that is nervous about it. The project fails when it is attempted as a big-bang rewrite with `strict` on from day one,

and succeeds when it is incremental, reversible and visibly delivering value within the first week.



Five stages, each independently valuable, each shippable within days.

Stage by stage

1. **Add TypeScript without converting anything.**
   Install `typescript`, add a `tsconfig.json` with `allowJs: true`, `checkJs: false`, `strict: false`, `noEmit: true`. Wire `tsc --noEmit` into CI. At this point it passes trivially — that is the point. The build is proven and nobody's workflow changed.

2. **Turn on `checkJs` for one directory**
   using a second tsconfig or `// @ts-check` pragmas. Fix what it finds — usually real bugs: typos in property names, functions called with the wrong arity, unhandled undefined. Ship those fixes as bug fixes, because that is what they are, and the team sees value before any migration cost.

3. **Install `@types` for every dependency**
   that has them. This alone transforms the editor experience and costs nothing.

4. **Rename leaf files first.**
   Start with modules nothing imports *from* — constants, pure utilities, type definitions. Renaming `.js` to `.ts` there cannot break anything above it. Work up the dependency graph. Use `madge` or `dependency-cruiser` to find the leaves.

5. **Define the domain types once**
   and reuse them. This is where the real benefit appears: a single `Order` type that every module agrees on, derived from a Zod schema at the API boundary.

6. **Enable strict flags one at a time**

, in this order: `noImplicitAny`, `strictNullChecks` (the big one — budget real time), `strictFunctionTypes`, `strictBindCallApply`, `noImplicitThis`, then the rest. Each flag is a separate pull request.

7 **Add the ratchet.**
A CI script that counts remaining `any`s, `@ts-expect-error`s and unchecked files and fails if the count increases. This is what makes the migration irreversible without requiring it to be finished.

JSON

```json
// tsconfig.migration.json - a permissive starting config
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "allowJs": true,
    "checkJs": false,
    "strict": false,
    "noImplicitAny": false,
    "noEmit": true,
    "skipLibCheck": true,
    "esModuleInterop": true,
    "resolveJsonModule": true
  },
  "include": ["src/**/*"]
}
```

BASH

```bash
#!/usr/bin/env bash
# scripts/type-ratchet.sh - fail CI if type debt increases
set -euo pipefail

BASELINE_FILE=".type-baseline.json"

ANY_COUNT=$(grep -rE ':\s*any\b|<any>|as any' src --include='*.ts' --include='*.tsx' |
IGNORE_COUNT=$(grep -rE '@ts-(ignore|expect-error|nocheck)' src | wc -l)
JS_COUNT=$(find src -name '*.js' -o -name '*.jsx' | wc -l)

BASE_ANY=$(jq -r '.any' "$BASELINE_FILE")
BASE_IGNORE=$(jq -r '.ignores' "$BASELINE_FILE")
BASE_JS=$(jq -r '.jsFiles' "$BASELINE_FILE")

fail=0
[ "$ANY_COUNT"    -gt "$BASE_ANY" ]    && { echo "any:    $ANY_COUNT > $BASE_ANY";
[ "$IGNORE_COUNT" -gt "$BASE_IGNORE" ] && { echo "ignore: $IGNORE_COUNT > $BASE_IGNORE
[ "$JS_COUNT"     -gt "$BASE_JS" ]     && { echo "js:     $JS_COUNT > $BASE_JS";

if [ "$fail" -eq 1 ]; then
  echo "Type debt increased. Fix it, or update $BASELINE_FILE with a justification."
  exit 1
fi

# Ratchet DOWN automatically when the numbers improve
jq -n --argjson a "$ANY_COUNT" --argjson i "$IGNORE_COUNT" --argjson j "$JS_COUNT" \
   '{any:$a, ignores:$i, jsFiles:$j}' > "$BASELINE_FILE"
echo "Type debt: any=$ANY_COUNT ignores=$IGNORE_COUNT js=$JS_COUNT"
```

The migration that fails

A long-lived `feature/typescript-migration` branch. It diverges from main, rebases become agony, feature work keeps landing on main, and after four months someone quietly closes it. Every single stage above merges within days. If a change cannot be merged this week, it is too big — split it. The second failure mode is turning on `strict` globally on day one, producing 4,000 errors, and demoralising the team into giving up.

Useful tools

Tool	What it does
`ts-migrate` (Airbnb)	Bulk-renames files and inserts `@ts-expect-error` where needed, giving you a compiling baseline to work down from
`typescript-strict-plugin`	Lets you mark individual files as strict with a comment, so you can go strict file by file
`type-coverage`	Reports the percentage of expressions with a non-`any` type — a single number to track on a dashboard
`madge` / `dependency-cruiser`	Finds leaf modules and circular dependencies so you can order the work
`@typescript-eslint`	Rules like `no-explicit-any`, `no-unsafe-assignment`, `ban-ts-comment` to hold the line
`quicktype` / `json-to-zod`	Generate types or Zod schemas from a sample JSON payload — fast way to type an undocumented API

Performance and Scale

On a small project TypeScript is instant. On a 3,000-file monorepo with heavy generic libraries, a cold type-check can take minutes and the editor can lag on every keystroke, which destroys the productivity you adopted it for. The causes are well understood and the fixes are mechanical.

Measuring before optimising

BASH

```
# Where is the time going?
npx tsc --noEmit --diagnostics
# Files, Lines, Nodes, Identifiers, Symbols, Types, Instantiations,
# Memory used, Parse/Bind/Check/Emit time.
# "Instantiations" in the millions means heavy generic/conditional type work.

# Which files and types are expensive?
npx tsc --noEmit --generateTrace ./trace
npx @typescript/analyze-trace ./trace
# Prints the slowest files and the specific type instantiations responsible.

# Is the editor slow for the same reason?
# VS Code: "TypeScript: Open TS Server log" and check for long requests.
```

The usual culprits

Cause	Symptom	Fix
`skipLibCheck` off	Slow cold builds; errors inside `node_modules`	Turn it on. Almost always correct.
Large union types (hundreds of members)	Slow assignability checks everywhere they appear	Use a branded string or an interface instead of an exhaustive literal union
Deep conditional/recursive types	Huge instantiation counts; "excessively deep" errors	Add depth guards; simplify; cache intermediate results in named types
Intersections of big object types	Slow, and terrible error messages	Use `interface extends` instead of `&` — interfaces are cached
No `incremental` / no project references	Every build is a full build	Enable `incremental`; split into projects
Barrel files (`index.ts` re-exporting everything)	Importing one symbol pulls in the whole graph	Import from the specific module; avoid deep barrel chains
Heavily generic libraries used naively	Per-call-site instantiation explosions	Annotate the result type explicitly to stop inference work

Project references

JSON

```jsonc
// tsconfig.json at the repo root - orchestrates, builds nothing itself
{
  "files": [],
  "references": [
    { "path": "./packages/shared" },
    { "path": "./packages/api" },
    { "path": "./packages/web" }
  ]
}
```

JSON

```jsonc
// packages/shared/tsconfig.json
{
  "extends": "../../tsconfig.base.json",
  "compilerOptions": {
    "composite": true,              // REQUIRED for a referenced project
    "declaration": true,
    "declarationMap": true,
    "rootDir": "src",
    "outDir": "dist",
    "tsBuildInfoFile": "dist/.tsbuildinfo"
  },
  "include": ["src/**/*"]
}

// packages/api/tsconfig.json
{
  "extends": "../../tsconfig.base.json",
  "compilerOptions": { "composite": true, "rootDir": "src", "outDir": "dist" },
  "references": [{ "path": "../shared" }],
  "include": ["src/**/*"]
}
```

BASH

```bash
tsc --build                     # builds only what changed, in dependency order
tsc --build --watch             # incremental watch across the whole graph
tsc --build --clean             # remove outputs and build info
tsc --build --force             # full rebuild when you suspect stale state
```

✦ **What project references buy you**

Each package is type-checked once and its `.d.ts` output is consumed by dependents, so changing a leaf package does not re-check the entire monorepo. On a large repo this routinely turns a 90-second check into 8 seconds. It also enforces the dependency graph — package A cannot secretly import from package B unless the reference exists, which is a genuine architectural benefit beyond speed. The cost is more configuration and the need to build before some editor features work across package boundaries.

`isolatedModules` and why bundlers need it

TYPESCRIPT

```
// With isolatedModules, each file must be transpilable ALONE.
// These are errors because a single-file transpiler cannot know the answer:

// 1. Re-exporting a type without the `type` keyword
export { SomeType } from "./types.js";          // ERROR
export type { SomeType } from "./types.js";     // correct

// 2. const enum (no cross-file inlining possible)
// const enum Direction { Up }                  // ERROR under isolatedModules

// 3. A file with no imports or exports is a script, not a module
// (add `export {}`)

// The upside: esbuild/SWC can transpile 1000 files in parallel with no
// type information at all, which is why modern builds are so fast.
```

Keeping types fast as the codebase grows

- [ ] `skipLibCheck: true` and `incremental: true` from day one.
- [ ] Prefer `interface X extends Y` over `type X = Y & Z` for object composition.
- [ ] Give intermediate type computations names; the compiler caches named types.
- [ ] Annotate exported function return types explicitly — it stops the compiler re-inferring them at every call site and improves `.d.ts` emit.
- [ ] Avoid barrel `index.ts` files in large packages; import from the concrete module.

- [ ] Cap template-literal unions; do not generate thousands of members.
- [ ] Put a depth guard on every recursive type.
- [ ] Run `--generateTrace` when a check gets slow rather than guessing.
- [ ] Split into project references once a single `tsc --noEmit` exceeds ~30 seconds.

Interview Questions

What is the difference between `any` and `unknown`, and when would you use each?

Both can hold any value, but they differ in what you are allowed to do with them. With `any`, the compiler stops checking entirely: you can access any property, call it, and assign it to anything, and the `any` spreads to every expression it touches, which is why one `any` can silently disable checking across a whole module. With `unknown`, you can assign anything *into* it but you cannot do anything *with* it until you narrow it — no property access, and it is not assignable to any other type. That makes `unknown` the correct type for data crossing a trust boundary: the result of `JSON.parse`, a `fetch` response body, a caught exception, or `process.env` values. You then prove the shape with a type guard or, better, a schema parse. I use `any` only as a deliberate temporary escape hatch during a migration, with a comment explaining why, and I configure the `no-explicit-any` lint rule so each one is a conscious decision rather than a habit.

TypeScript types do not exist at runtime. What are the practical consequences?

The main one is that no data entering your program is actually validated by the type system — it is validated by your assertion, which may be wrong. Writing `const user = await res.json() as User` tells the compiler to trust you, and if the API returns a number where you declared a string, or omits a field, nothing fails at that line. The failure surfaces much later, somewhere unrelated, as "cannot read properties of undefined", and the stack trace points at the victim rather than the cause. The consequence for design is that you need runtime validation at every boundary: HTTP responses, request bodies, environment variables, localStorage, message queue payloads, and database rows, since drivers return loosely typed objects. The pattern I use is to define a Zod schema and derive the type with `z.infer`, so there is exactly one definition and the check and the type cannot drift. Secondary consequences: you cannot do `if (x instanceof MyInterface)`, you cannot reflect over types to generate a form or an ORM mapping without a build step, and generics are erased so you cannot branch on `T` at runtime.

Explain discriminated unions and why they are better than optional properties.

A discriminated union is a union of object types that all share a literal-typed field — `kind`, `type`, `status` — with a different value in each member. TypeScript uses that field to narrow: inside `if (state.status === "success")` the compiler knows exactly which member you have and gives you

its specific fields with no cast. The alternative people reach for is one interface with everything optional — `loading?`, `data?`, `error?` — and that is worse for two reasons. First, it permits nonsense states like loading and error being simultaneously true, which means every consumer has to write defensive logic and they will not all agree on the precedence. Second, every access needs a null check even in branches where the value is logically guaranteed, so the code is noisier and the compiler helps less. The phrase I use is "make illegal states unrepresentable". The additional payoff is exhaustiveness: with a `default` branch that assigns the value to `never`, adding a new member to the union causes every switch that does not handle it to fail compilation, which turns a scary refactor into a to-do list from the compiler.

What does `strict` actually turn on, and which flag matters most?

`strict` is an umbrella for about eight flags: `noImplicitAny`, `strictNullChecks`, `strictFunctionTypes`, `strictBindCallApply`, `strictPropertyInitialization`, `noImplicitThis`, `useUnknownInCatchVariables` and `alwaysStrict`. By far the most important is `strictNullChecks`. Without it, `null` and `undefined` are assignable to every type, which means the single largest category of JavaScript runtime errors — reading a property of undefined — is completely invisible to the type system. With it on, a value that might be absent has type `T | null` and the compiler forces you to handle it before use. The others matter too: `noImplicitAny` stops untyped parameters silently becoming `any`; `useUnknownInCatchVariables` types caught errors as `unknown`, which is correct since JavaScript can throw anything. Beyond `strict` I also enable `noUncheckedIndexedAccess`, which makes array and record access return `T | undefined` — initially noisy, but it catches a real class of off-by-one and missing-key bugs. On a legacy codebase I would enable these one flag at a time, each as its own pull request, rather than all at once.

When would you use `interface` versus `type`?

They overlap for object shapes, so the honest answer is that consistency matters more than the choice, but there are two real differences. Only `type` can express unions, intersections of non-objects, tuples, primitives, mapped types, conditional types and template literal types — so anything derived or computed must be a `type`. Only `interface` supports declaration merging, where you reopen it in another file and add members; that is essential for augmenting third-party or global types, such as adding a `user` property to Express's `Request` or a field to `Window`. There are also minor practical differences: interfaces tend to produce better error messages because the name is preserved rather than the structure expanded, and they are marginally faster for the compiler on very large shapes because the resolved members are cached. My convention is `interface` for object shapes that form a public API or might need augmenting, and `type` for everything else. I would not re-litigate an existing codebase's convention over this.

How do generics work and when should you reach for one?

A generic introduces a type parameter that is filled in at the call site, letting you express a relationship between types rather than fixing them. The classic example is `function first<T>(arr: T[]): T | undefined`, where the return type is tied to the element type of the input, so calling it with a `User[]` gives you `User | undefined` with no cast. Constraints restrict what the parameter can be — `T`

`extends { length: number }` lets you access `.length` inside — and `K extends keyof T` is the pattern behind typed property access and `Pick`. The test I apply before adding a type parameter is whether it appears at least twice, typically once in the parameters and once in the return type, or twice in the parameters to force two arguments to agree. If `T` appears exactly once, the generic is doing nothing that `unknown` or a constraint would not do, and it just makes the signature harder to read. Beyond that there is a second tier — conditional types, `infer`, mapped types — which you rarely need to write but constantly need to read, because that is what library types are made of.

How would you migrate a large JavaScript codebase to TypeScript?

Incrementally, with every stage merged to main within days, because the long-lived migration branch is the way these projects die. First I add TypeScript with `allowJs`, `checkJs: false`, `strict: false` and `noEmit`, and wire `tsc --noEmit` into CI — it passes trivially, which proves the pipeline works and changes nobody's workflow. Then I install `@types` packages for dependencies, which immediately improves the editor for the whole team at zero cost. Then I enable `checkJs` on one directory, or add `// @ts-check` pragmas with JSDoc types, and fix what it finds — which is usually genuine bugs like misspelled properties and wrong arity, shipped as bug fixes so the team sees value before paying migration cost. Only then do I start renaming files, beginning with leaves that nothing imports from and working up the dependency graph. Finally I enable strict flags one at a time, `noImplicitAny` first and `strictNullChecks` as its own substantial piece of work. Throughout, I add a CI ratchet that counts remaining `any`s, `ts-expect-error`s and `.js` files and fails if any count increases, because you do not need to finish the migration — you need to guarantee it cannot go backwards.

What is `satisfies` and what problem does it solve?

It checks a value against a type without widening the value to that type. Before `satisfies`, you had two bad options. Annotating — `const config: Config = { ... }` — validates the shape but throws away the specific literal types, so `config.port` becomes `string | number | boolean` and you cannot call a number method on it. Using `as const` keeps the literal types but performs no validation at all, so a typo in a key or a disallowed value type goes unnoticed. `satisfies` gives you both: the expression is checked against the type and any excess or wrong property is an error at the declaration, but the inferred type remains the specific literal shape. The most common real use is a lookup table — a route map, a theme, a config object — where you want `keyof typeof routes` to be the exact union of keys and the values to be the exact literal strings, while still guaranteeing every value matches a constraint like `` `/${string}` ``. You can also combine it with `as const` for a deeply readonly, checked, literal-typed object.

A colleague writes `const user = await res.json() as User`. What is wrong with it?

It is an unverified assertion dressed up as a type. `res.json()` returns `Promise<any>`, and the cast tells the compiler to stop asking questions — no check happens at runtime. If the API returns `{"id": 1}` where `id` is declared as a string, or renames `name` to `fullName`, or returns `null` for a field declared non-nullable, this line succeeds and the program continues with a value that violates its own type. The failure then appears somewhere else entirely, often in a UI component, as "cannot read

properties of undefined", and debugging starts at the wrong end of the call chain. The fix is to type the parsed JSON as `unknown` and run it through a schema: `const user = UserSchema.parse(await res.json())`, with `type User = z.infer<typeof UserSchema>` so the schema is the single source of truth. That throws immediately, at the boundary, with a message naming the exact field and the exact problem. The broader principle is that every `as` on data from outside your program is a bug you have not encountered yet, and the whole point of validating at the edge is that everything inside can then trust its types completely.

How do you type errors in TypeScript?

You cannot type what a function throws — there is no `throws` clause and the language deliberately does not track it, because JavaScript's `throw` is unconstrained. Under `strict`, `catch (e)` gives you `unknown`, which is correct since any value can be thrown, so the first thing in any catch block is narrowing: `if (e instanceof Error)`, or a normaliser function that converts anything into an `Error`. Beyond that I use two complementary approaches. For unexpected failures — programmer errors, infrastructure problems — I use a class hierarchy with a literal `code` field on each subclass, which makes them a discriminated union so a single top-level handler can switch on the code and map to an HTTP status exhaustively. For expected, domain-level failures — not found, validation failed, a business rule violated — I prefer returning a `Result` type, a union of `{ ok: true, value }` and `{ ok: false, error }`, because that puts the failure in the signature where the compiler forces the caller to handle it. The trade-off is verbosity and friction with libraries that throw, so I draw the line at "is this part of the domain or is it exceptional".

Your team's `tsc --noEmit` takes four minutes and the editor lags. How do you diagnose it?

I start with `tsc --noEmit --diagnostics` to see the basic shape: file count, type count, and especially instantiation count, since instantiations in the millions point at heavy generic or conditional type work rather than simply having a lot of code. Then `--generateTrace` with `@typescript/analyze-trace`, which names the specific files and type instantiations consuming the time. The usual findings, in rough order of frequency: `skipLibCheck` is off so the compiler is checking every `.d.ts` in `node_modules`; a recursive or deeply conditional utility type with no depth guard; large intersections of big object types, which are recomputed rather than cached — switching to `interface extends` often helps measurably; enormous union types, sometimes generated accidentally by a template literal type; and barrel `index.ts` files that drag the entire module graph into every import. Structural fixes are enabling `incremental` and splitting the repo into project references, which type-checks each package once and reuses its declaration output, commonly taking a ninety-second check down to under ten seconds. I would measure before and after each change rather than applying all of them blind.

What are the downsides of TypeScript, honestly?

Several, and being able to name them is a sign you actually use it. Build complexity: you now have a compile step, a tsconfig with options that interact in non-obvious ways, and module resolution that can produce errors that look like type errors but are configuration problems. Error messages: a mismatch deep inside a generic produces a forty-line message that is genuinely hard to read until you

> learn to read the last line first. The false sense of safety is the most dangerous one — a fully typed codebase that casts at every boundary has all the ceremony and none of the protection, and teams routinely believe they are safe when they are not. There is a real learning curve past the basics; conditional types and `infer` are a second language. Type-level code can get clever to the point of being unmaintainable, and a sufficiently baroque utility type is worse than an `any` with a comment. Compile times on large repos need active management. And some patterns are genuinely more awkward than in plain JavaScript, particularly highly dynamic code and some metaprogramming. On balance I think it is clearly worth it for anything multi-person and long-lived, but I would not use it for a sixty-line script, and I would not pretend the costs are zero.

Checklist and Practice Plan

TypeScript is learned by hitting errors and understanding why, not by reading about types. The exercises below are ordered so each one forces a specific concept, and the checklists are what to apply on a real codebase.

Practice plan — eight focused exercises

1. **Model a state machine.**
   Take a request lifecycle — idle, loading, success, error, retrying — and model it as a discriminated union. Write a render function with an exhaustive switch and a `never` default. Then add a sixth state and watch exactly which code fails to compile. This is the single most important exercise.

2. **Build a typed API client.**
   Pick any public API. Write Zod schemas for three endpoints, derive the types with `z.infer`, and build a generic `request<T>(path, schema)` function. Then deliberately change a schema to the wrong type and confirm you get a precise error naming the field.

3. **Write five utility types from scratch**
   without looking them up: `MyPartial`, `MyPick`, `MyOmit`, `MyReturnType`, `MyAwaited`. Then write `DeepPartial` and `Paths` with a depth guard. This is the fastest way to internalise mapped and conditional types.

4. **Brand your identifiers.**
   In an existing project, convert `userId`, `orderId` and one more identifier to branded types. Count how many places the compiler finds where the wrong one was being passed. There is usually at least one.

5. **Type a generic React component.**

Build a `<DataTable>` where column accessors are constrained to `keyof T` or a render function, and prove that a typo in a column key is a compile error.

6. **Fix a real error chain.**
Find a forty-line "not assignable" error in a codebase and work out the mismatch from the last line alone, without reading the middle. Repeat until it takes under a minute.

7. **Migrate a small JS project.**
Take a 20-file JavaScript project through all five migration stages in one sitting. Note which stage produced the most real bugs — it is almost always `checkJs` with JSDoc, before any renaming.

8. **Profile a slow build.**
Run `--generateTrace` on any non-trivial project, read the analysis, and find the single most expensive type. Then make it cheaper and measure the difference.

Checklist: starting a new TypeScript project

- [ ] `strict: true`, plus `noUncheckedIndexedAccess` and `exactOptionalPropertyTypes`.
- [ ] `skipLibCheck: true`, `incremental: true`, `isolatedModules: true`, `verbatimModuleSyntax: true`.
- [ ] `module` / `moduleResolution` matched to the runtime or bundler, and verified by actually running the output.
- [ ] `tsc --noEmit` as a separate CI step, because your bundler does not type-check.
- [ ] ESLint with `@typescript-eslint`: `no-explicit-any`, `no-floating-promises`, `no-misused-promises`, `ban-ts-comment` allowing only `expect-error` with a description.
- [ ] A validation library at every boundary, with types derived from schemas.
- [ ] Environment variables validated at startup; the process exits with a readable message if they are wrong.
- [ ] A `Prettify` helper in the shared types file.
- [ ] Path aliases wired in *both* tsconfig and the bundler/runtime, with the output verified.

Checklist: code review

- [ ] Any `as` on data from outside the program — should be a schema parse.
- [ ] Any `any` without a comment explaining why and a plan to remove it.
- [ ] `@ts-ignore` anywhere — should be `@ts-expect-error` with a reason and a link.
- [ ] Non-null assertions (`!`) where a guard would be clearer and safer.
- [ ] A union being handled without an exhaustive `never` default.
- [ ] Optional properties used where a discriminated union expresses the real invariants.
- [ ] A type parameter that appears only once — it wants to be a constraint or `unknown` .
- [ ] Object literals typed by annotation where `satisfies` would preserve the literals.
- [ ] Database rows or API responses cast rather than parsed.
- [ ] A recursive type without a depth guard.
- [ ] Exported functions without explicit return types in a large codebase.

Checklist: handing a TypeScript codebase to a customer's team

- [ ] Domain types defined once, derived from schemas, in a single shared module.
- [ ] Every external boundary validated; no casts on untrusted data anywhere.
- [ ] `tsc --noEmit` and lint both green, enforced in CI, with no skipped checks.
- [ ] A type ratchet in place so the debt cannot grow after you leave.
- [ ] A README that states the module resolution setup and how to run the type check locally.
- [ ] No `any` in the public surface of any shared package.
- [ ] `attw` clean if you are publishing a package.
- [ ] Build time measured and documented, with project references in place if it exceeds about thirty seconds.